Repaso de los conceptos de metaprogramación

En donde repasamos los conceptos que vimos y consideramos características deseables para un sistema reflexivo.

Universidad Nacional de Quilmes

Pattie Maes

CONCEPTS AND EXPERIMENTS IN COMPUTATIONAL REFLECTION

Pattie Maes

AI-LAB
Vrije Universiteit Brussel
Pleinlaan 2
B-1050 Brussels
pattie@arti.vub.uucp

ABSTRACT. This paper brings some perspective to various concepts in computational reflection. A definition of computational reflection is presented, the importance of computational reflection is discussed and the architecture of languages that support reflection is studied. Further, this paper presents a survey of some experiments in reflection which have been performed. Examples of existing procedural, logic-based and rule-based languages with an architecture for reflection are briefly presented. The main part of the paper describes an original experiment to introduce a reflective architecture in an object-oriented language. It stresses the contributions of this language to the field of object-oriented [illegible] illustrates the new programming [illegible] examples show [illegible]

Reflective architectures have already been realised for procedure-based (Smith,1982), logic-based (Weyhrauch, 1980) and rule-based languages (Davis,1982). The second part of the paper (sections 6 to 8) discusses the realisation of a reflective architecture in an object-oriented language (further on called OOL). Existing OOLs only support limited, ad-hoc reflective facilities, which leads to limitations and unclear designs, and consequently to problems in [illegible] However, over the years OOL [illegible] providing more [illegible]

Gilad Bracha

David Ungar

Mirrors: Design Principles for Meta-level Facilities of Object-Oriented Programming Languages

Gilad Bracha
Sun Microsystems
4140 Network Circle
Santa Clara, CA 95054
(408) 276-7025
gilad.bracha@sun.com

David Ungar
Sun Microsystems
2600 Casey Ave., MTV 29-XXX
Mountain View, CA 94043
(650) 336-2618
david.ungar@sun.com

ABSTRACT

We identify three design principles for reflection and metaprogramming facilities in object oriented programming languages. *Encapsulation:* meta-level facilities must encapsulate their implementation. *Stratification:* meta-level facilities must be separated from base-level functionality. *Ontological correspondence:* the ontology of meta-level facilities should correspond to the ontology of the language they manipulate. Traditional/mainstream reflective architectures do not follow these precepts. In contrast, reflective APIs built around the concept of *mirrors* are characterized by adherence to these three principles. Consequently, mirror-based architectures have significant advantages with respect to distribution, deployment and general purpose metaprogramming.

Categories and Sub

D.3.2 [

```
Car anotherCar = theCarsClass.newInstance();
Class theCarsSuperclass = theCarsClass.getSuperclass();
```

Looking at the APIs of such a system, we expect to see something like:

```
class Object {
Class getClass();
...
}
class Class {
Class getSuperclass();
// many other methods: getMeth
}
```

Definición de comportamiento para objetos específicos

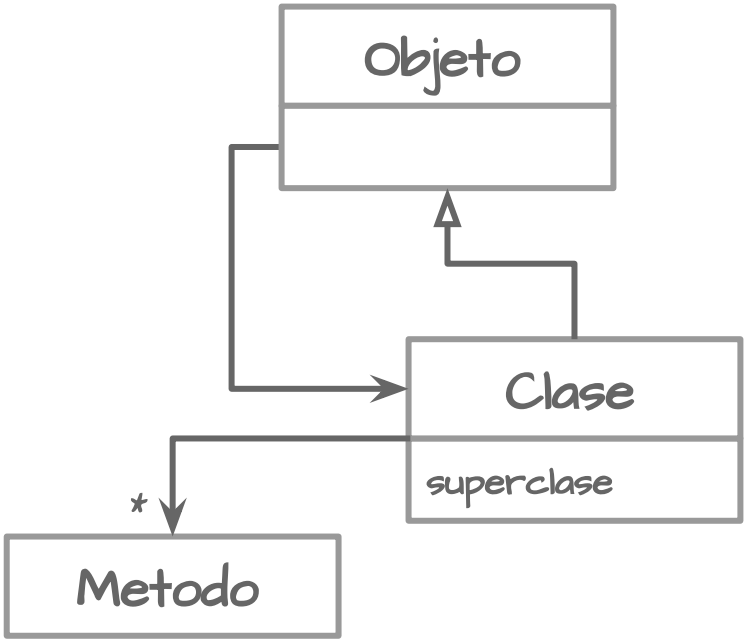
En donde revisamos el metamodelo de Ruby y vemos qué es una *singleton class*.

Universidad Nacional de Quilmes

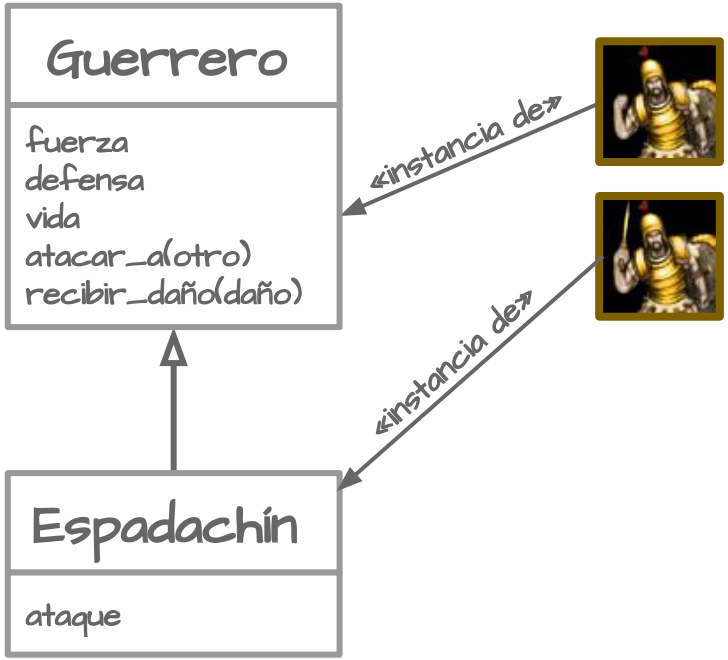
Repaso

Modelos y metamodelos

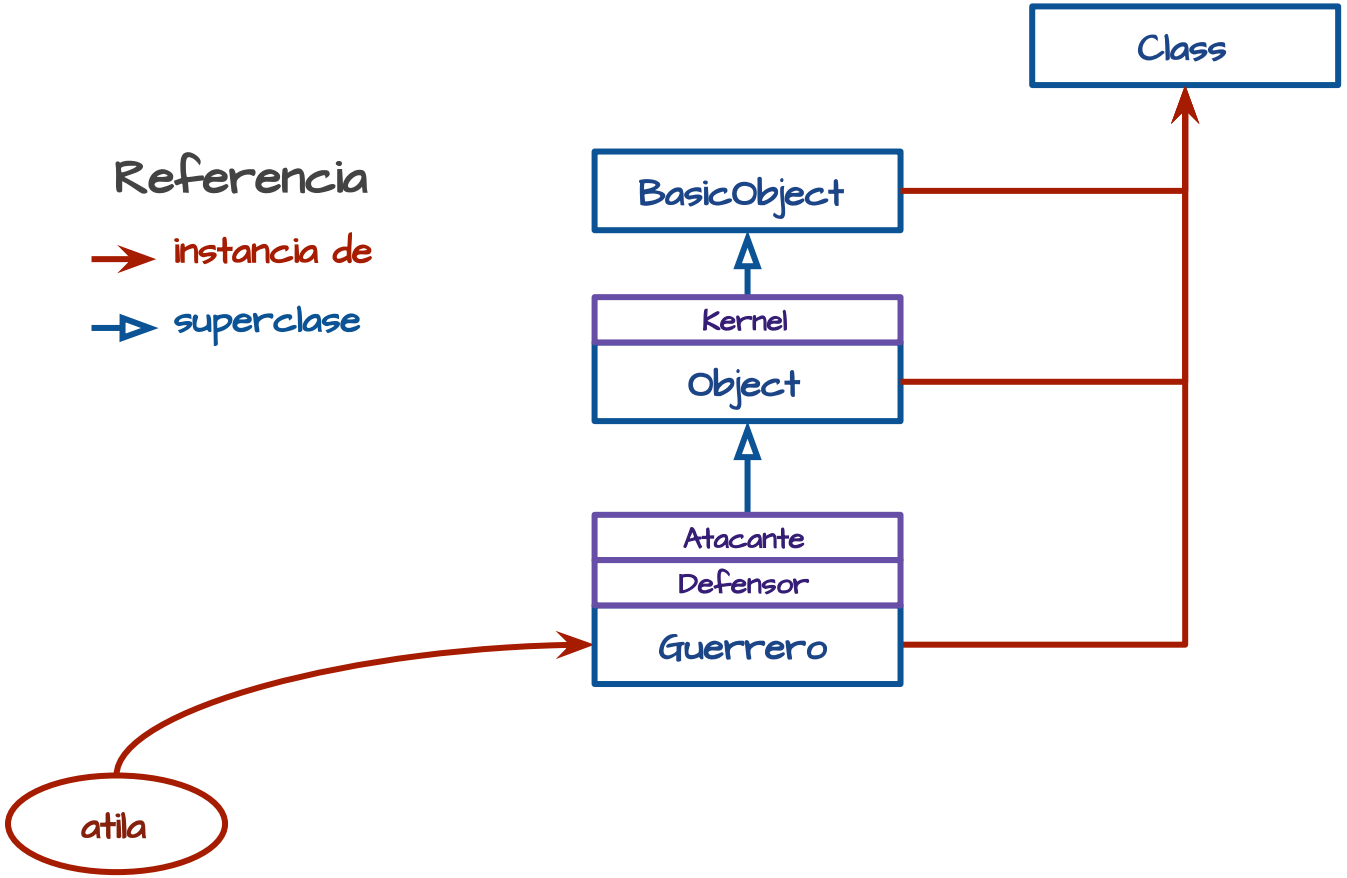
Metamodelo



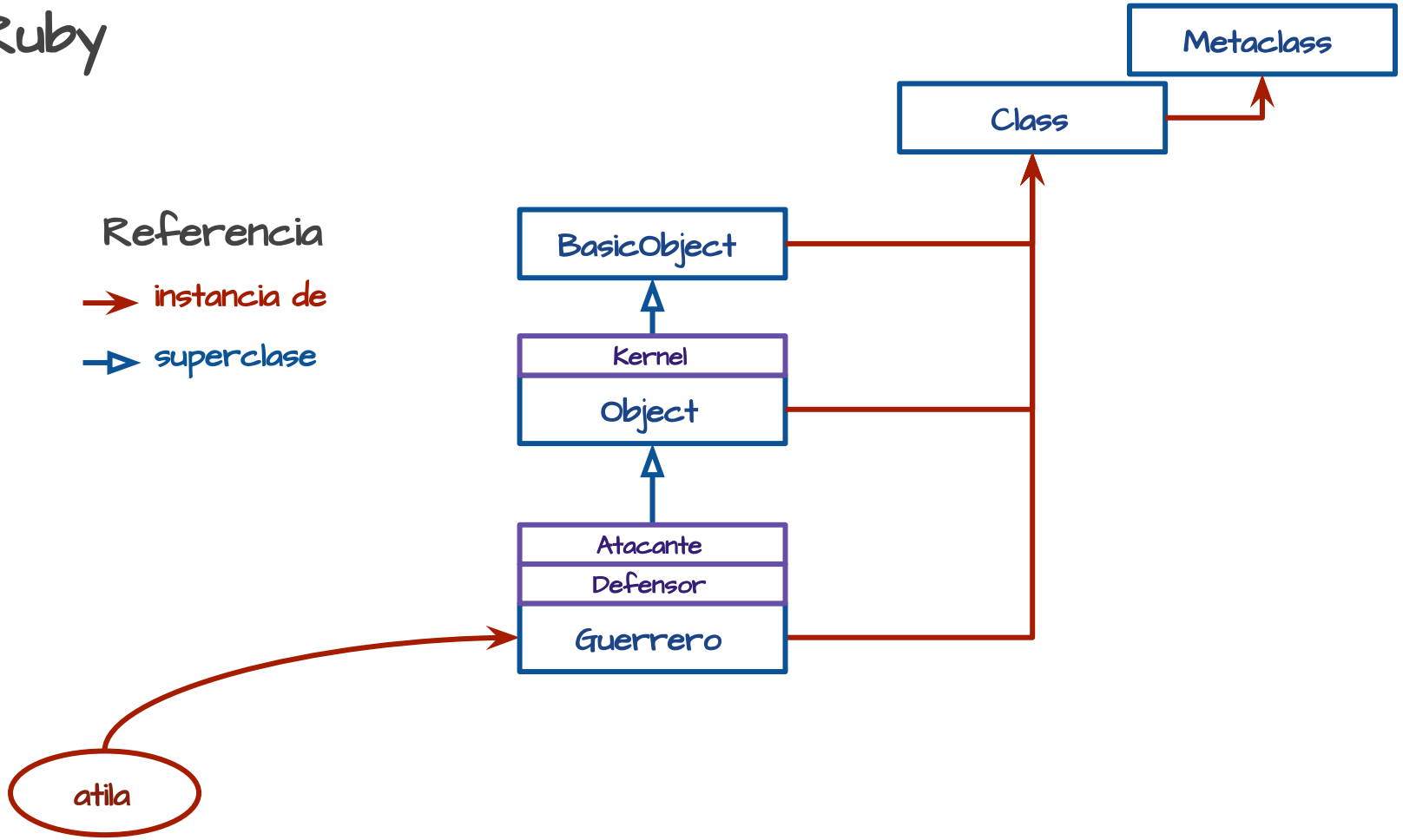
Modelo



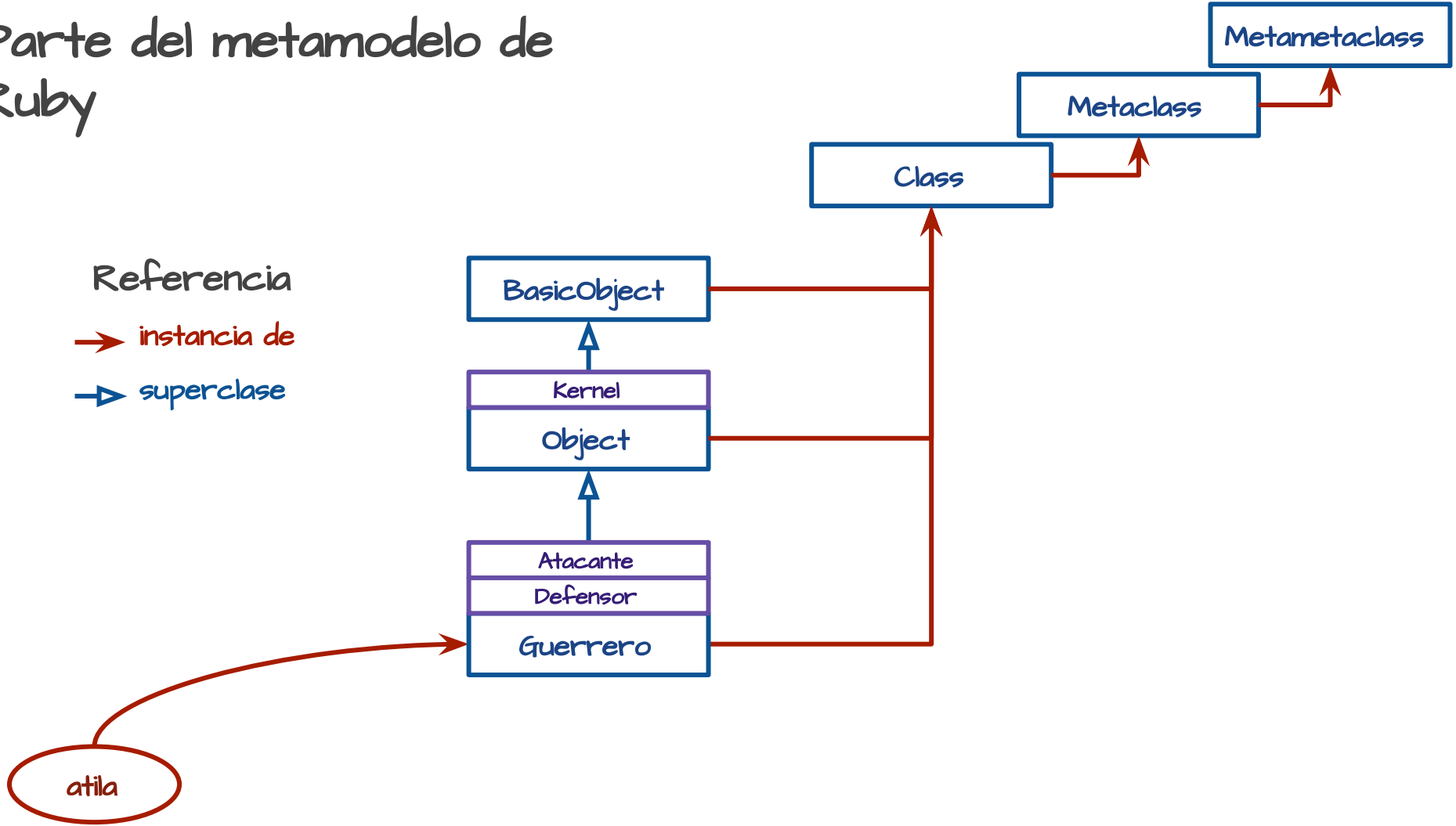
Parte del metamodelo de Ruby



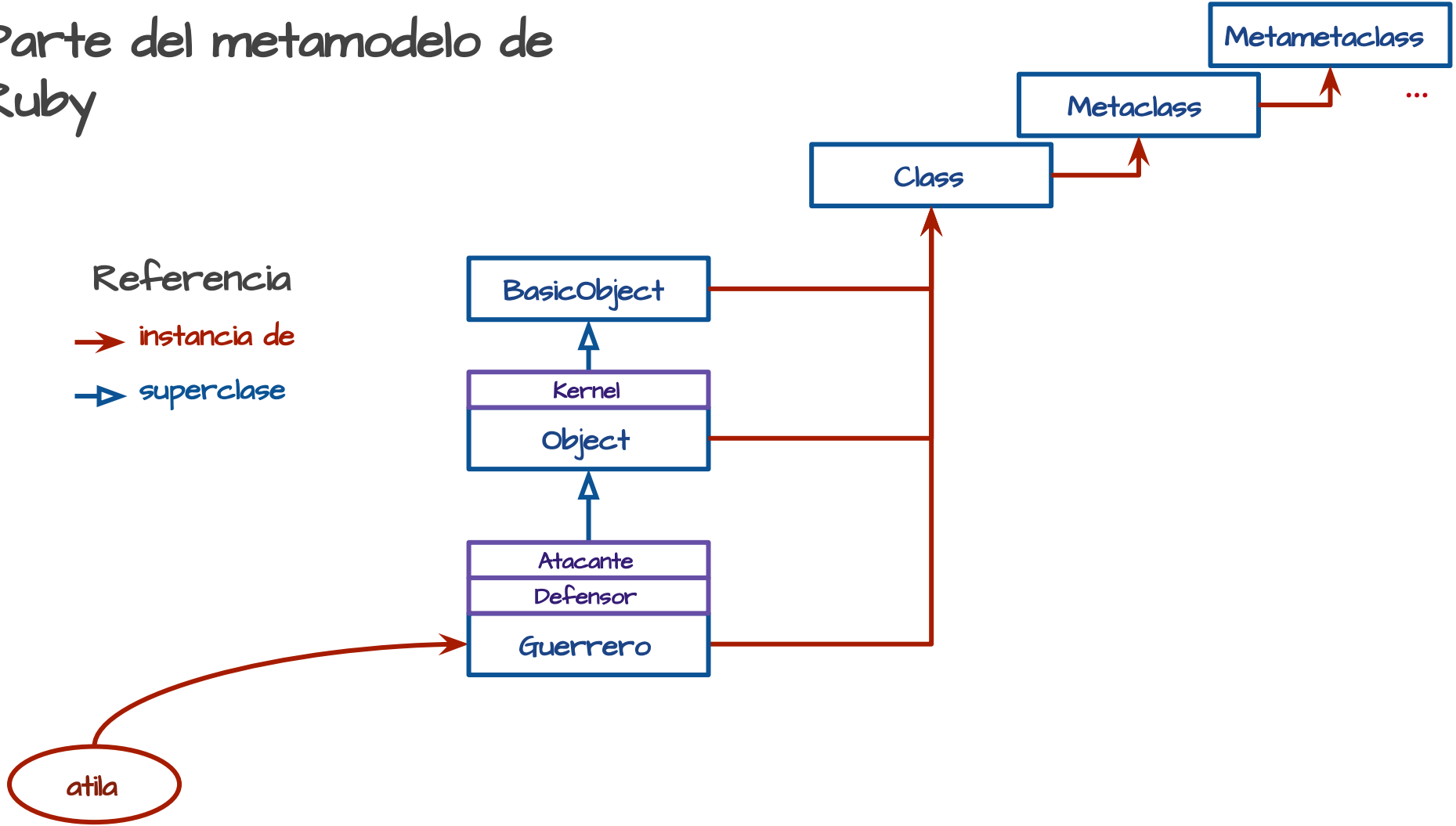
Parte del metamodelo de Ruby



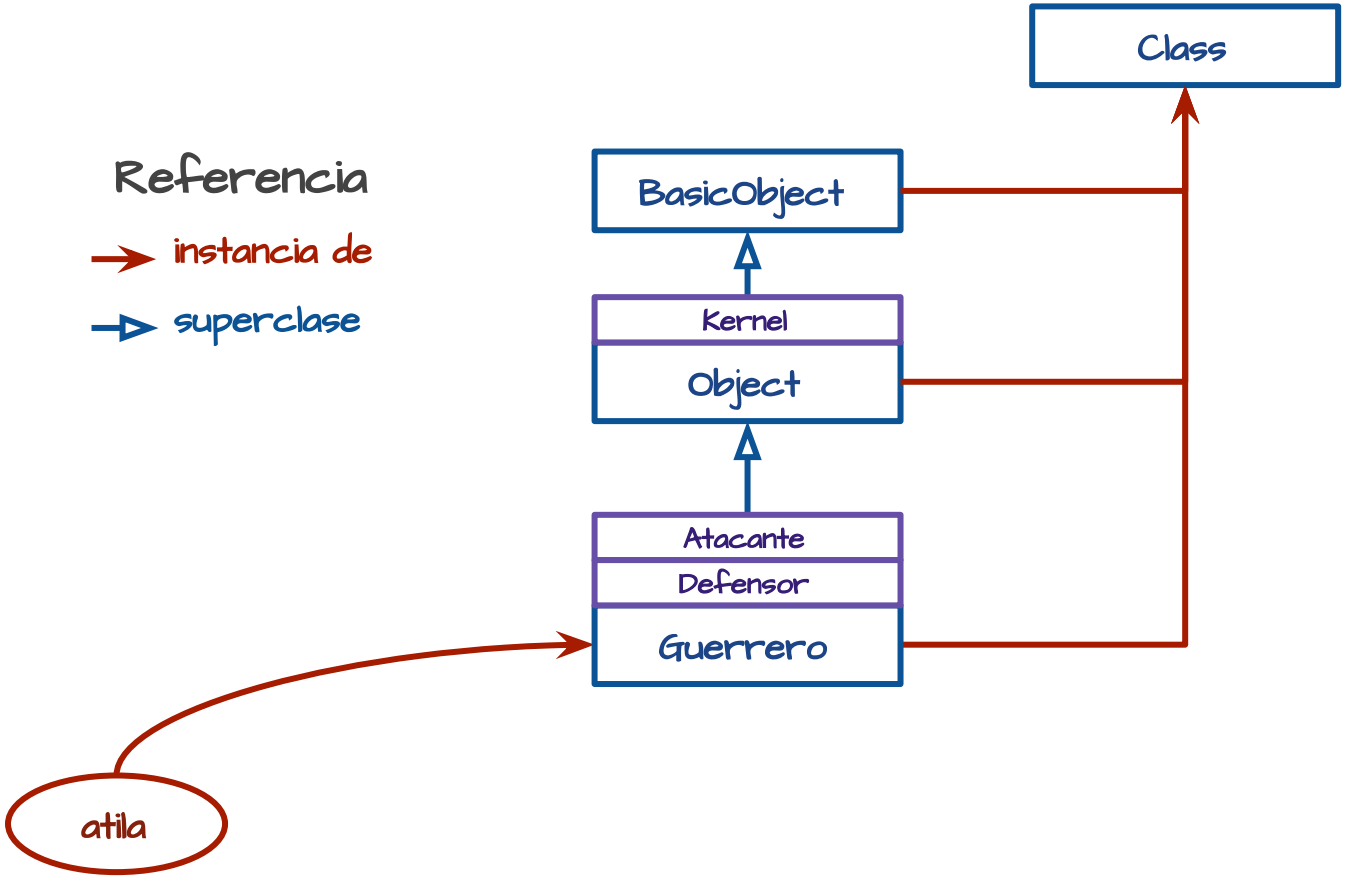
Parte del metamodelo de Ruby



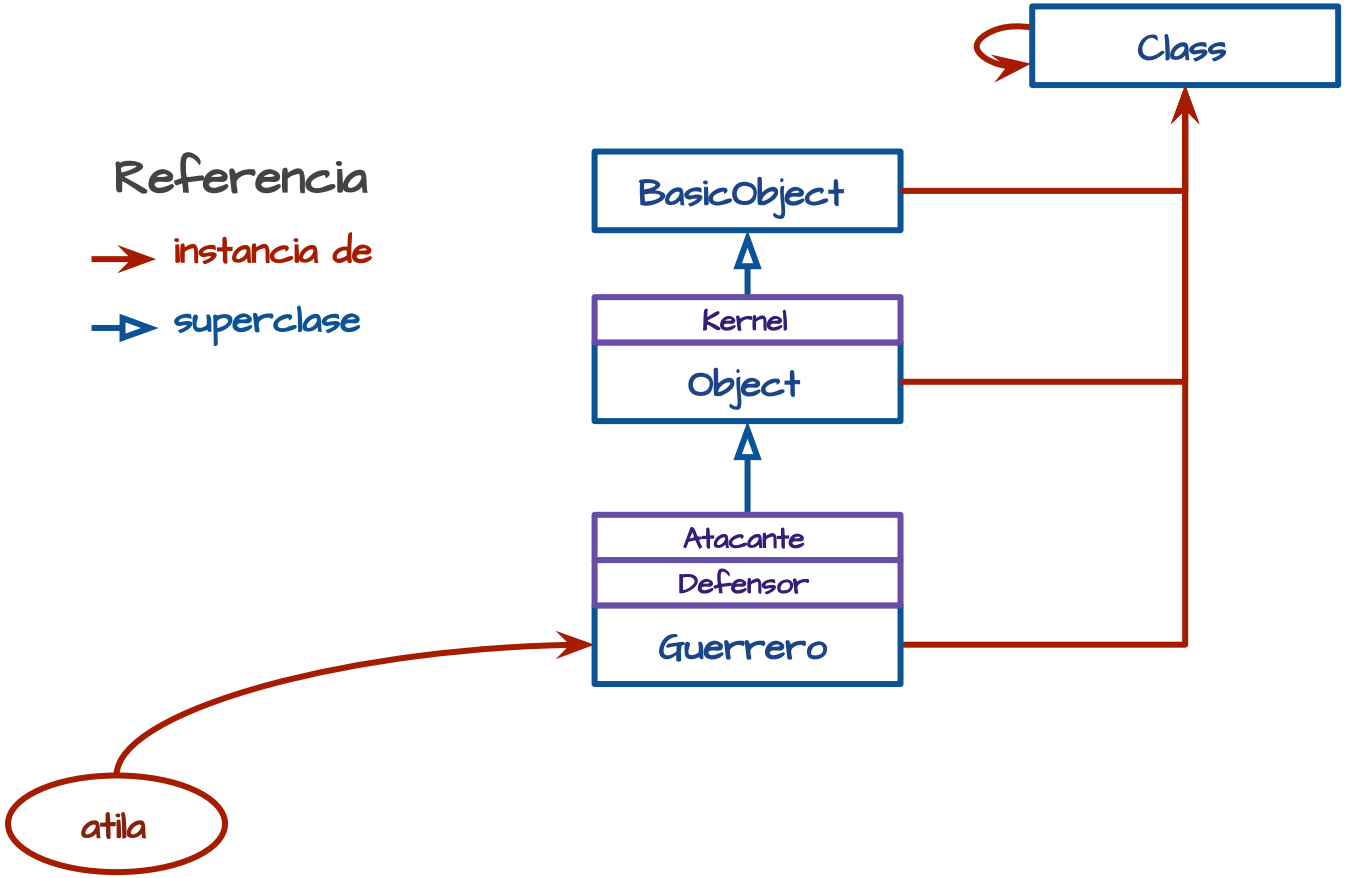
Parte del metamodelo de Ruby



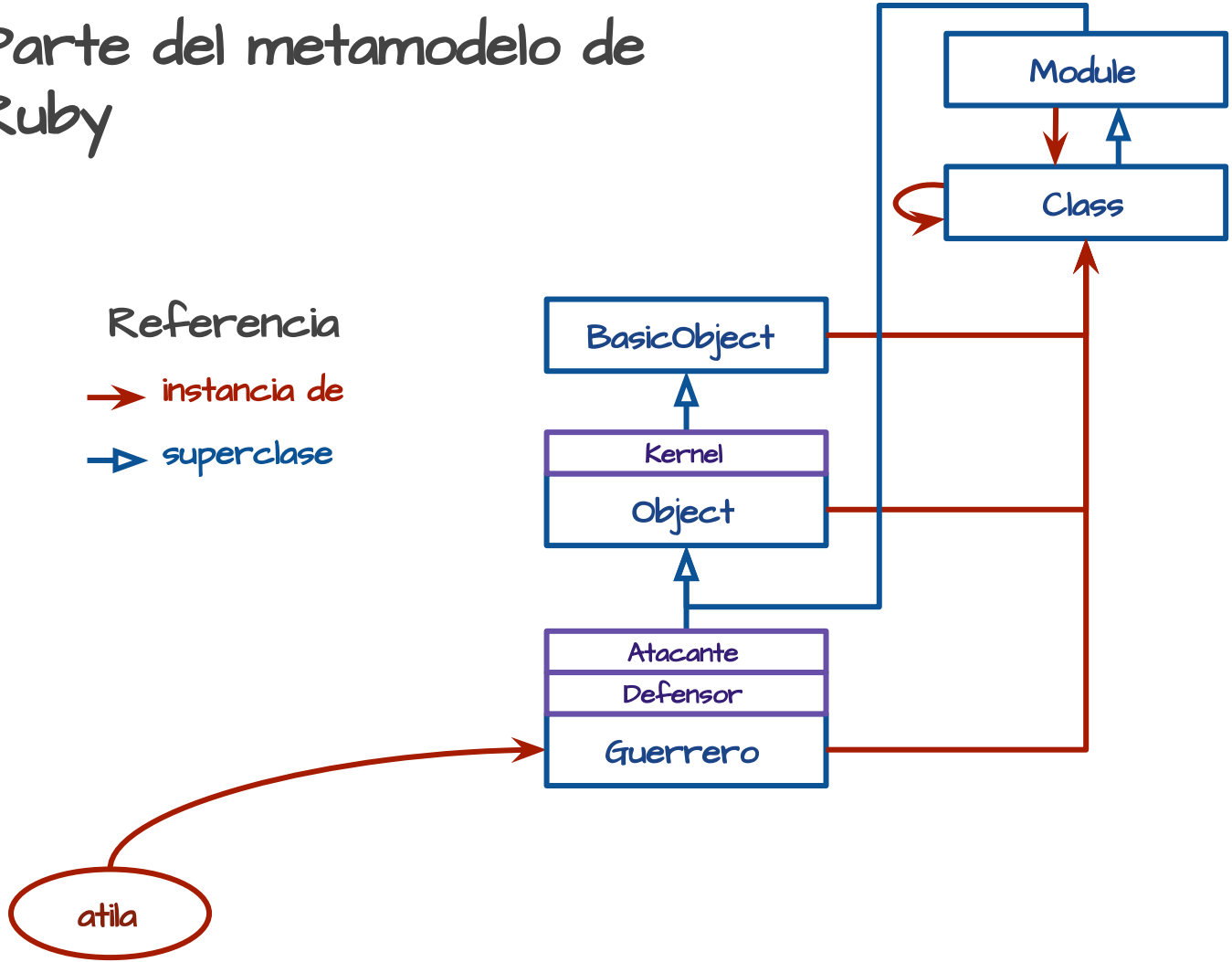
Parte del metamodelo de Ruby



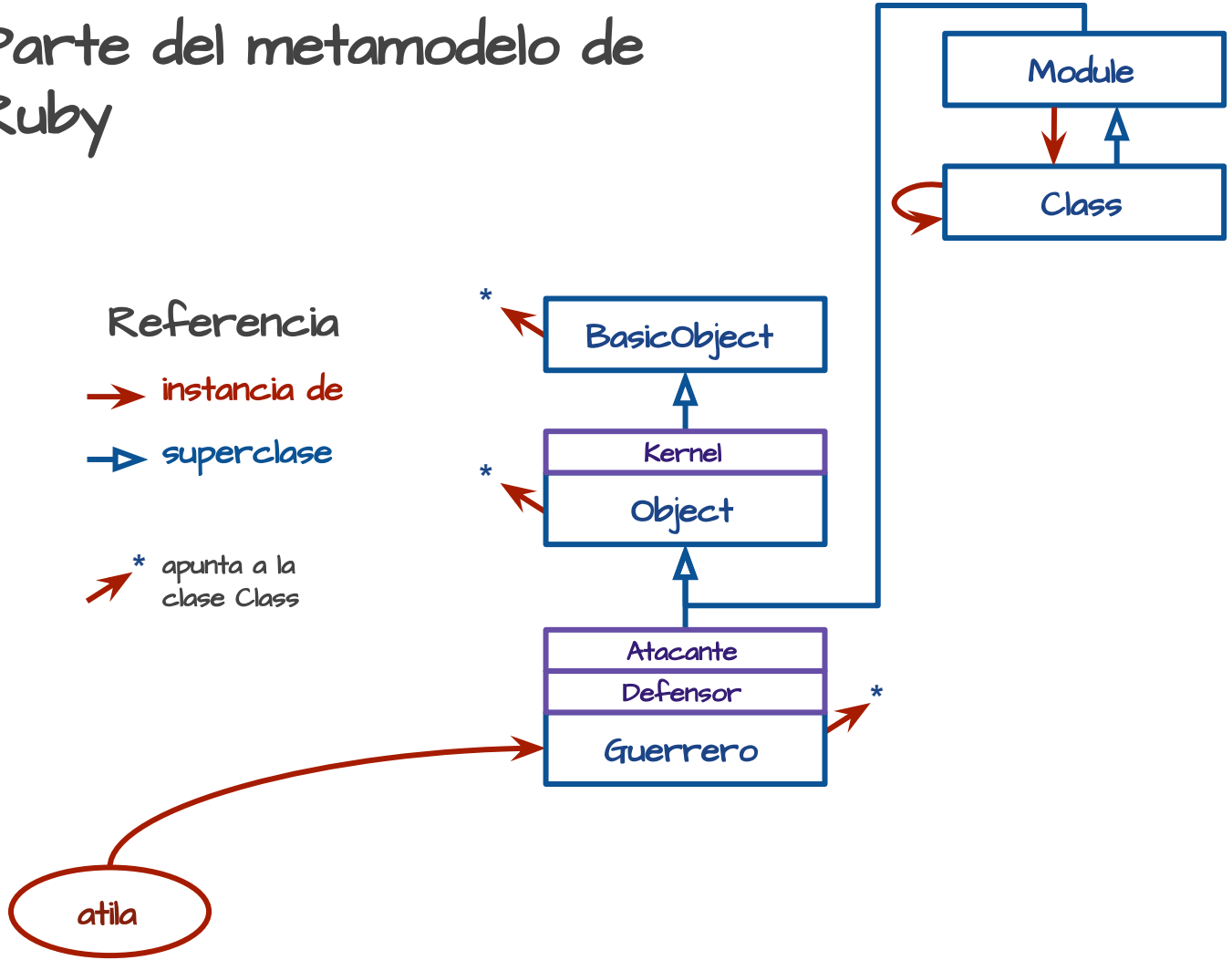
Parte del metamodelo de Ruby



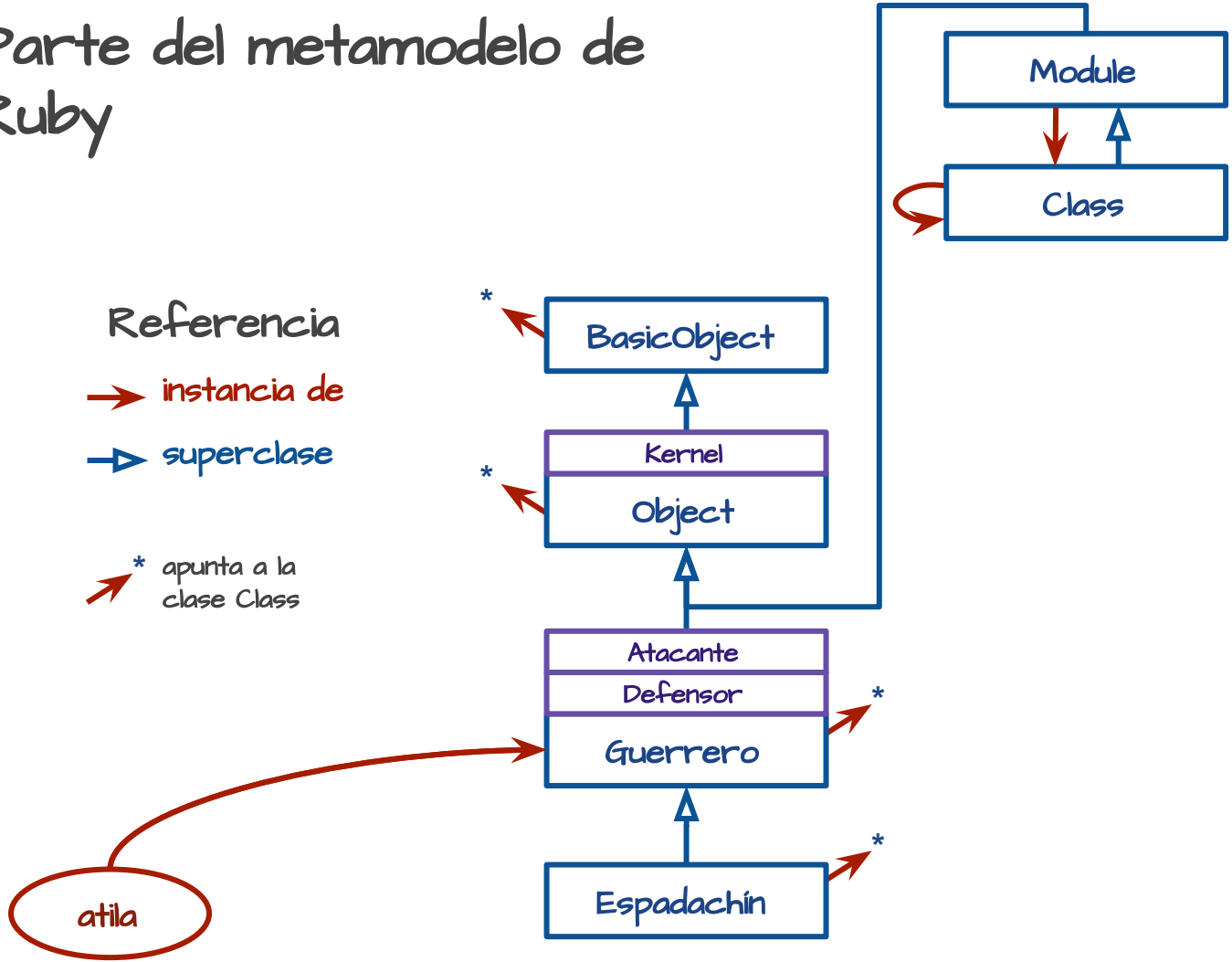
Parte del metamodelo de Ruby



Parte del metamodelo de Ruby

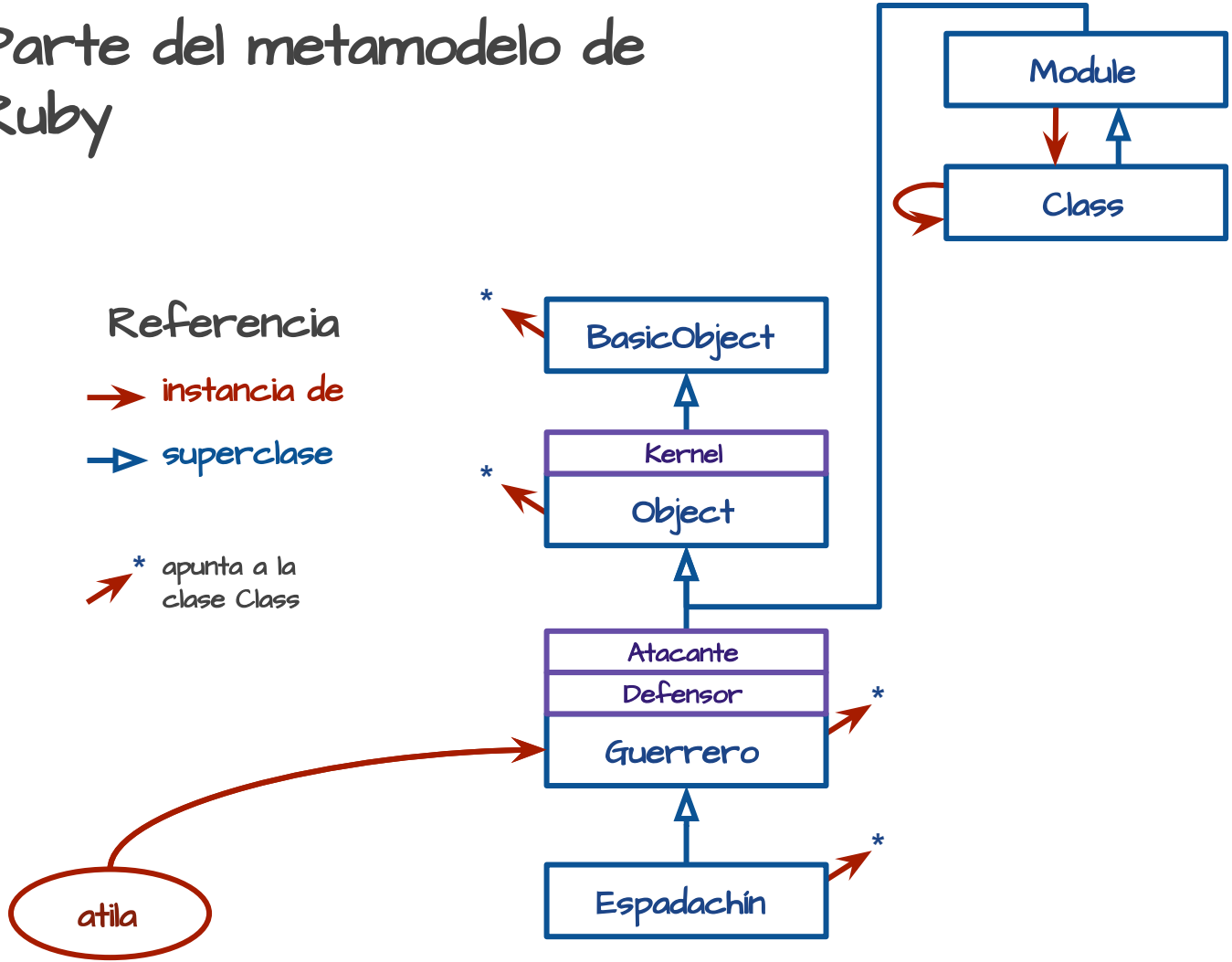


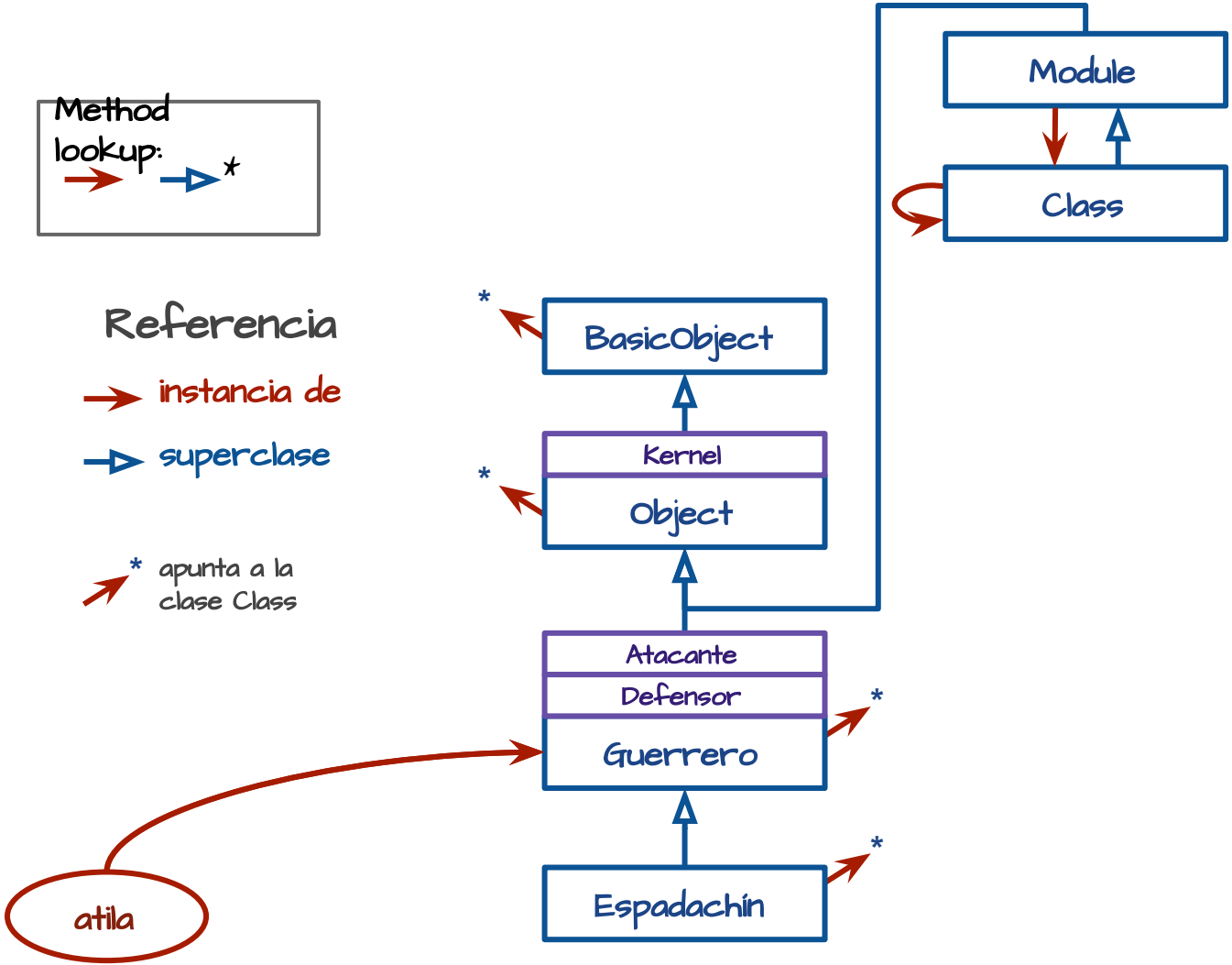
Parte del metamodelo de Ruby



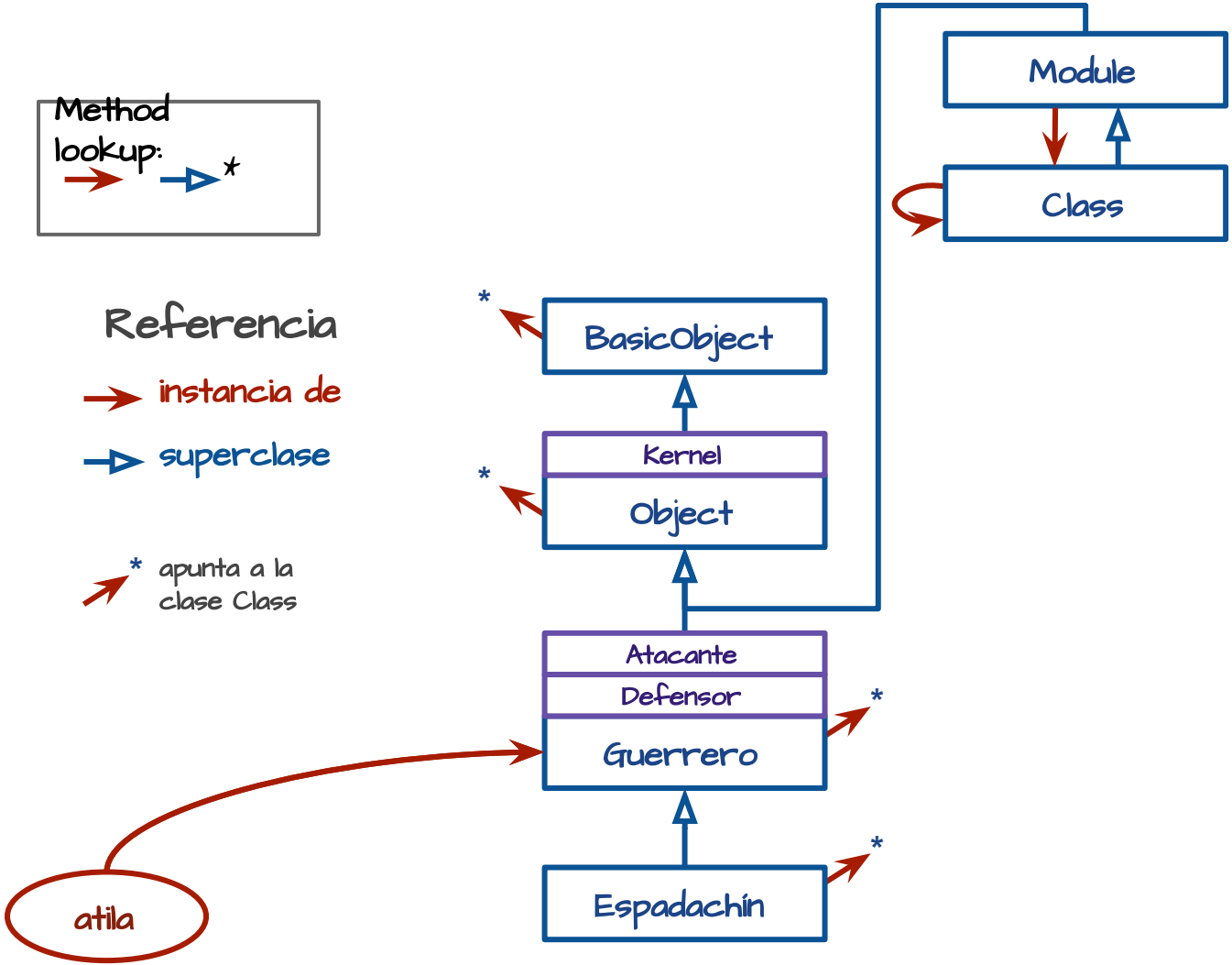
Repaso: el proceso de búsqueda de métodos

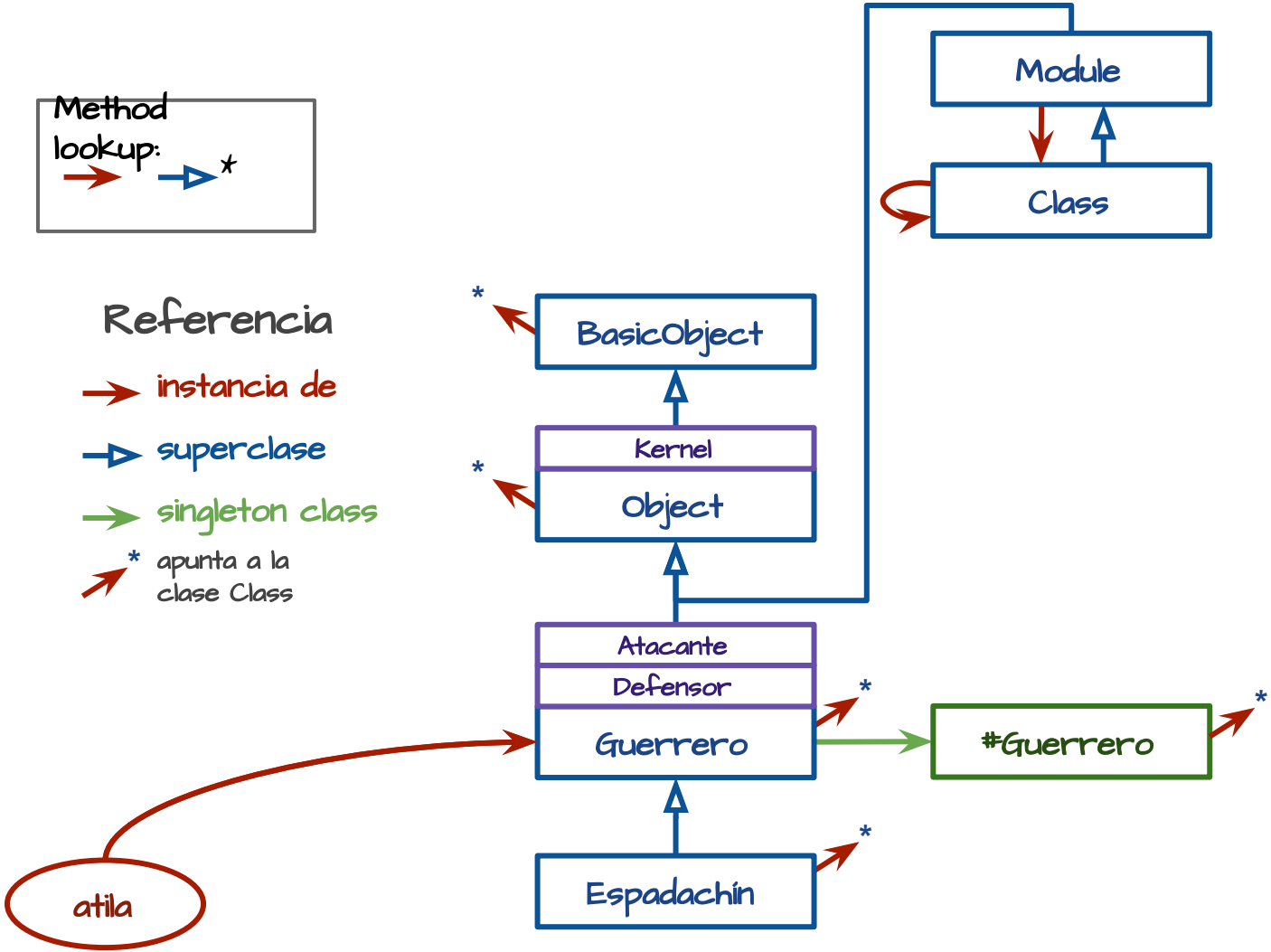
Parte del metamodelo de Ruby

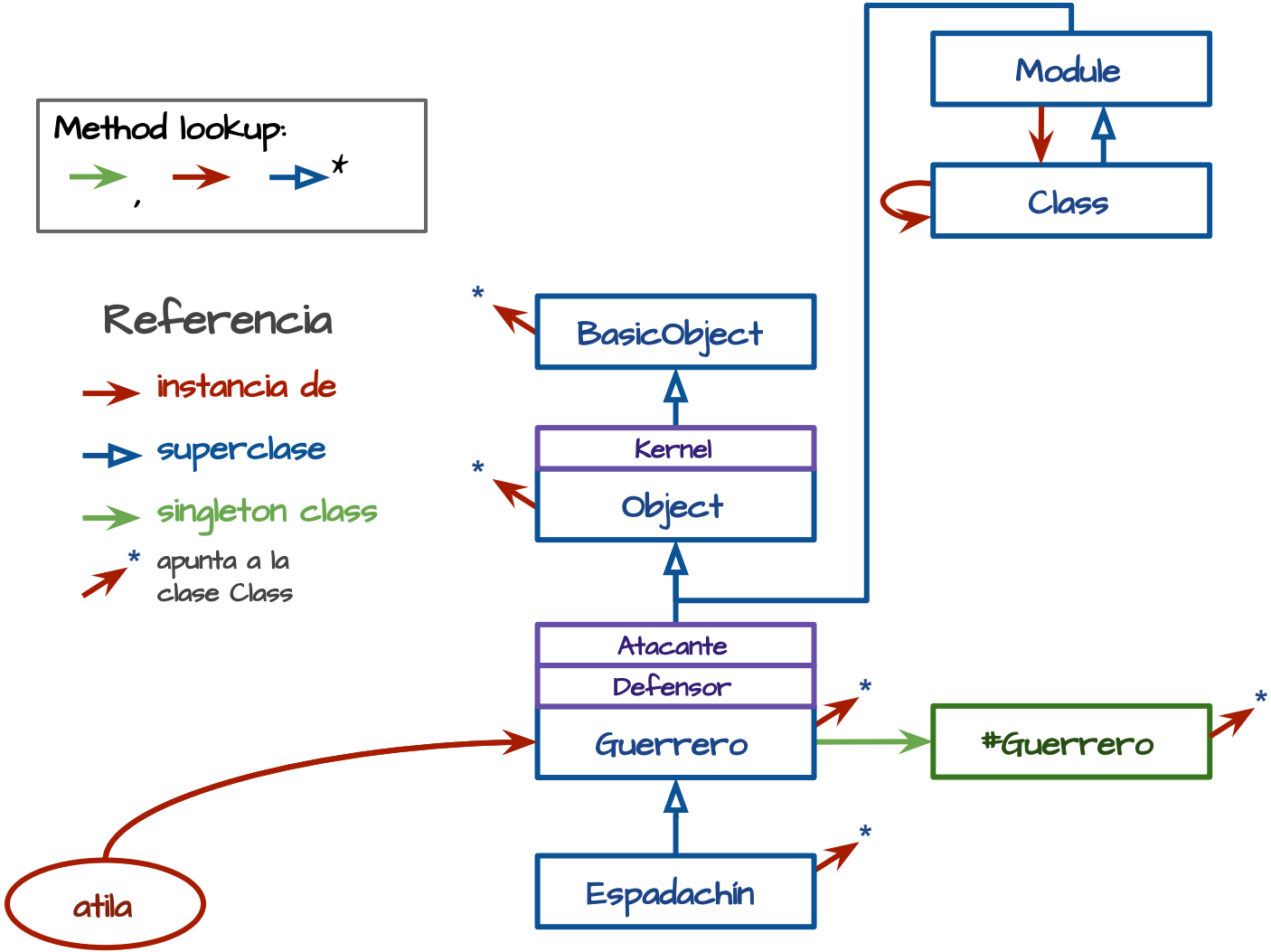


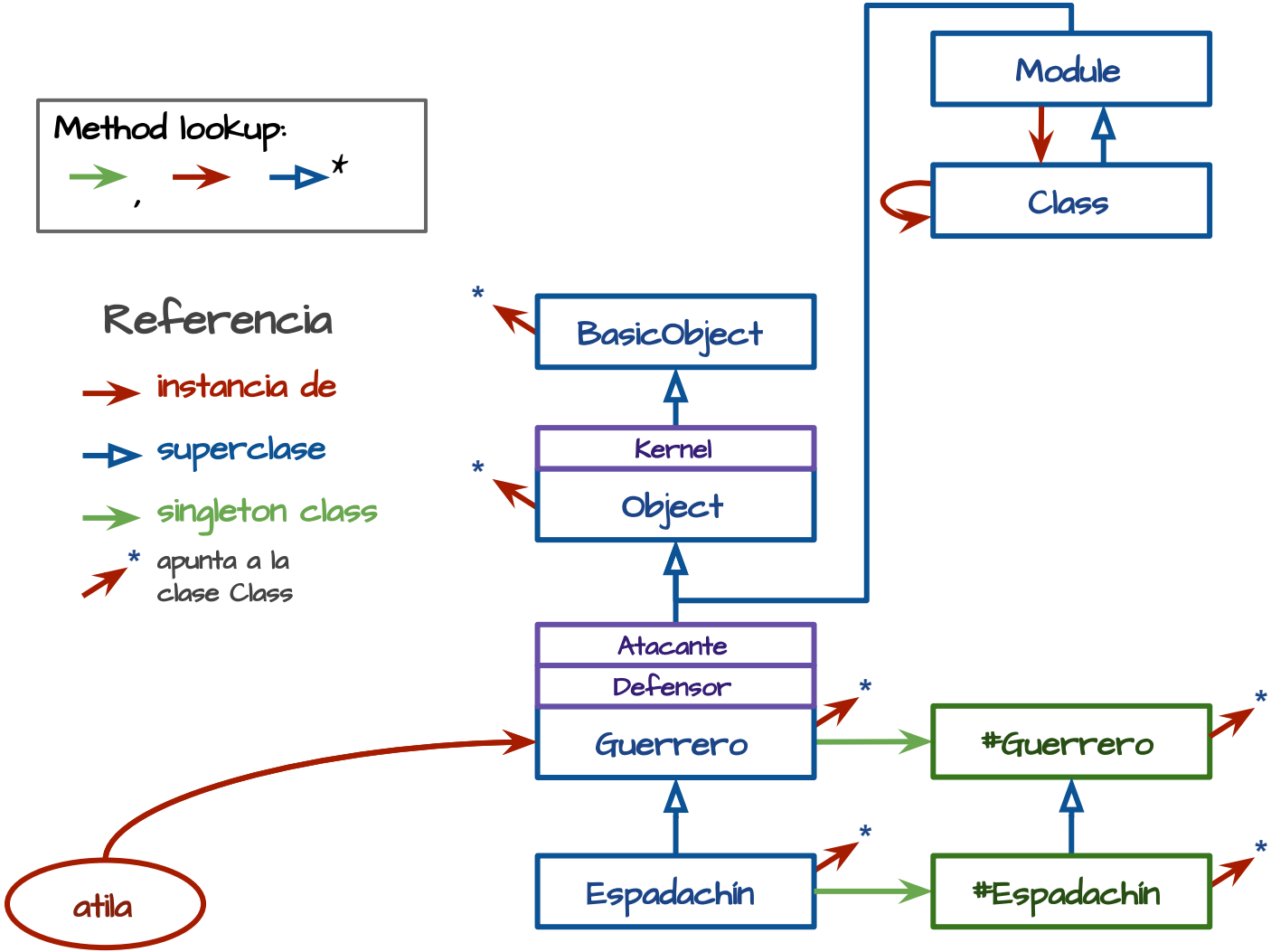

Method lookup:
*
Referencia
instancia de
superclase
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
Atacante
Defensor
Guerrero
Espadachín
atila

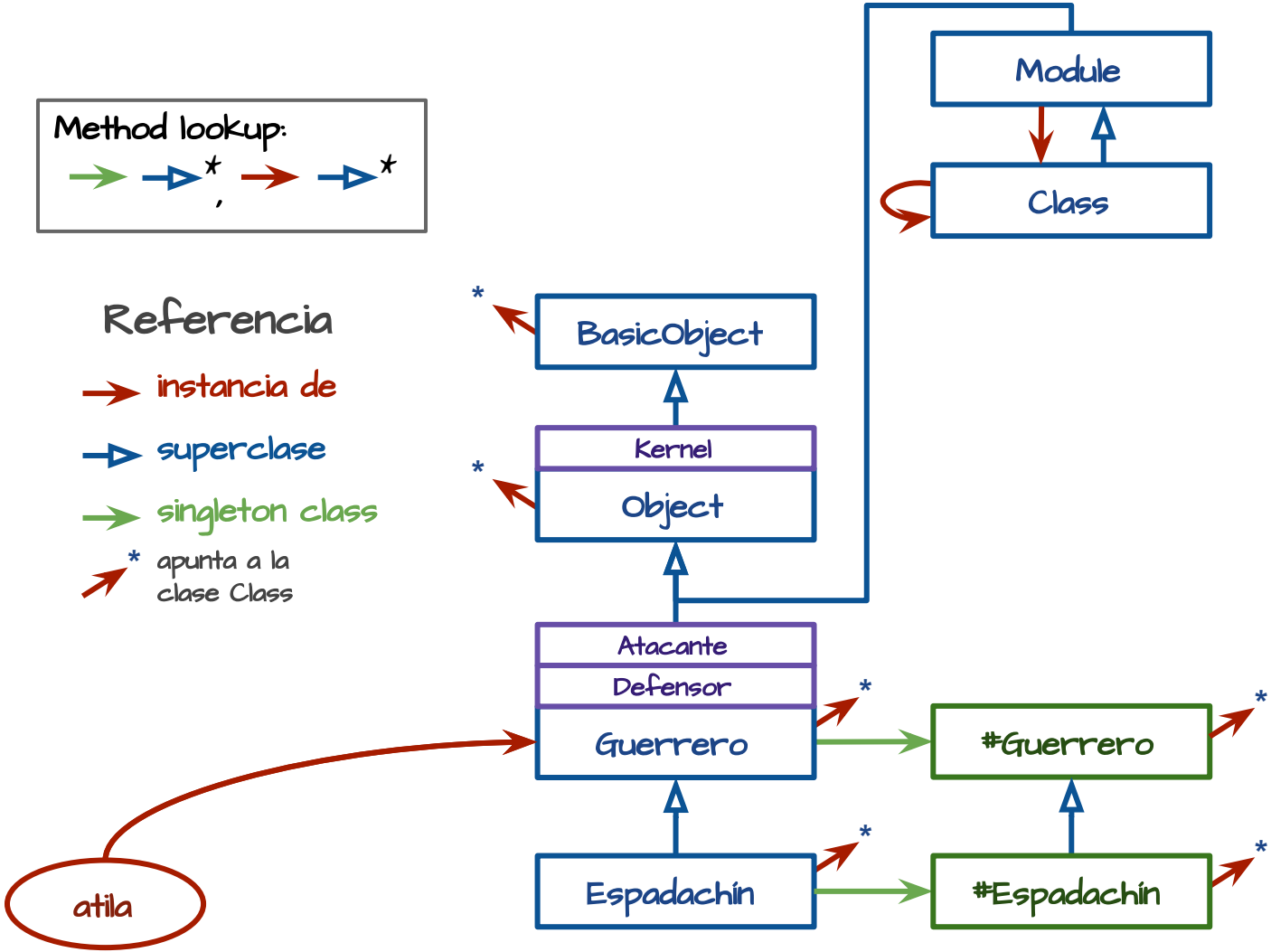
¿Cómo hacen las clases para entender mensajes específicos (“mensajes de clase”) en Ruby?

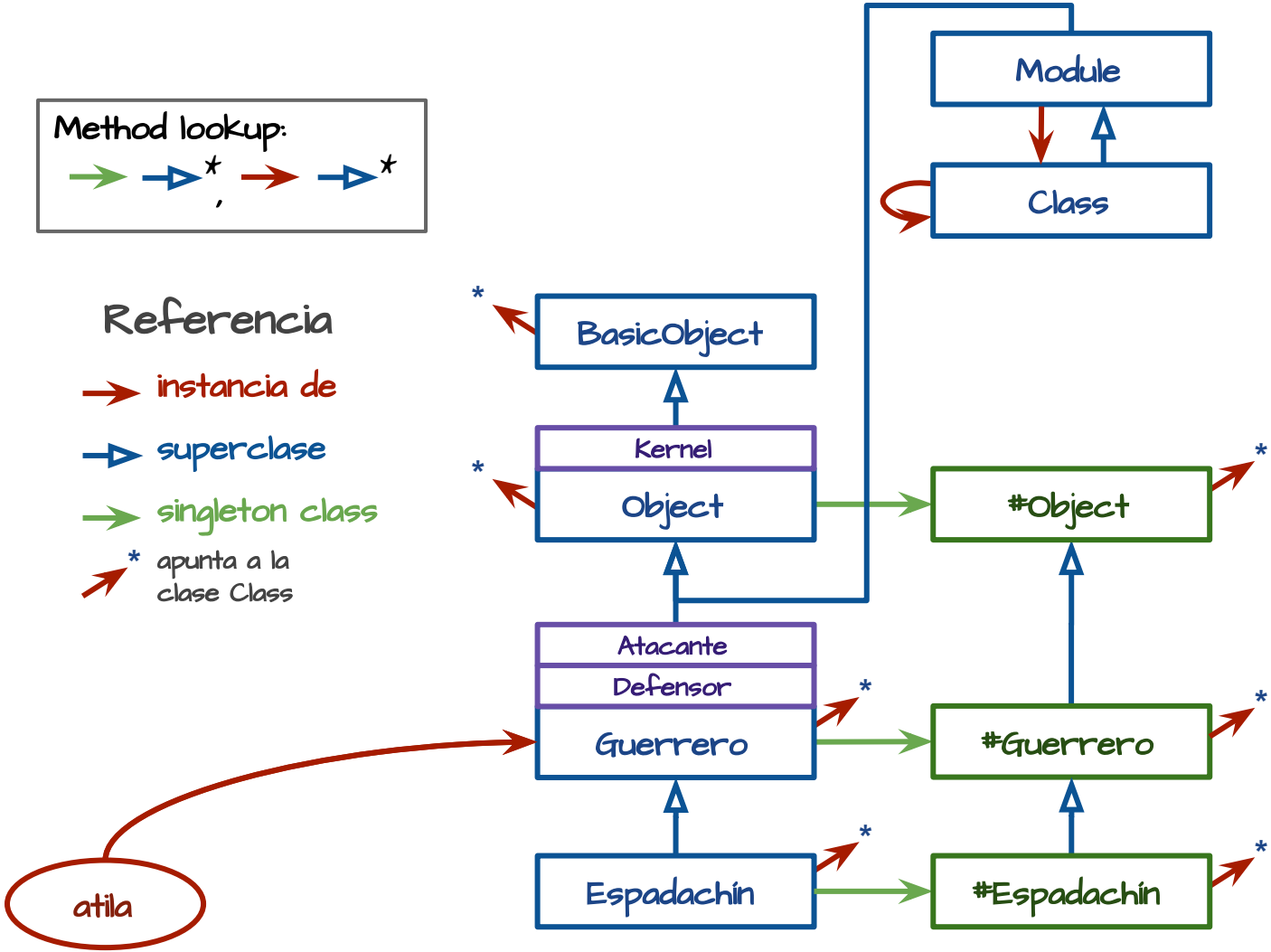

Method lookup:
*
Referencia
instancia de
superclase
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
Atacante
Defensor
Guerrero
Espadachín
atila

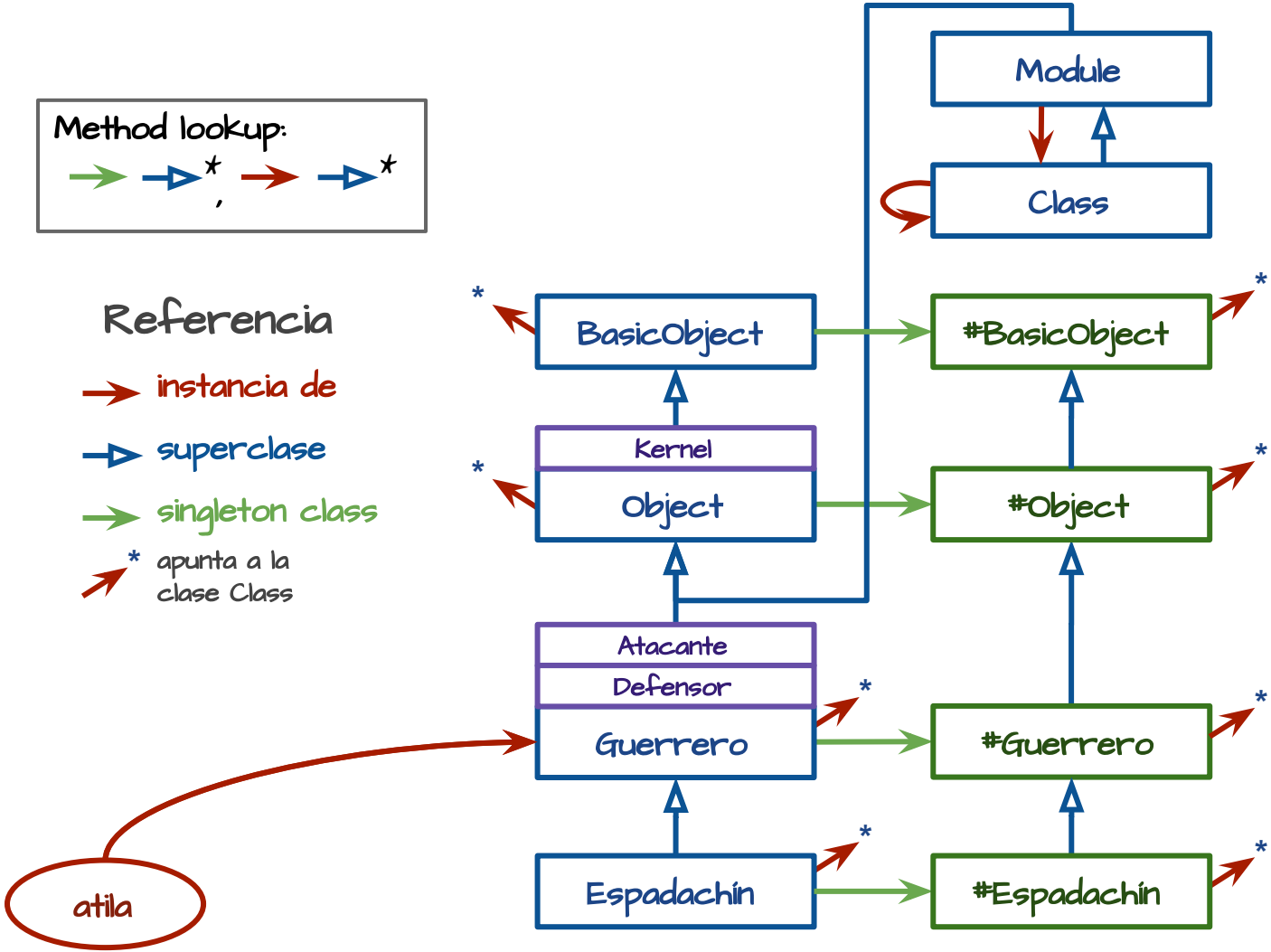

Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
atila


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
atila

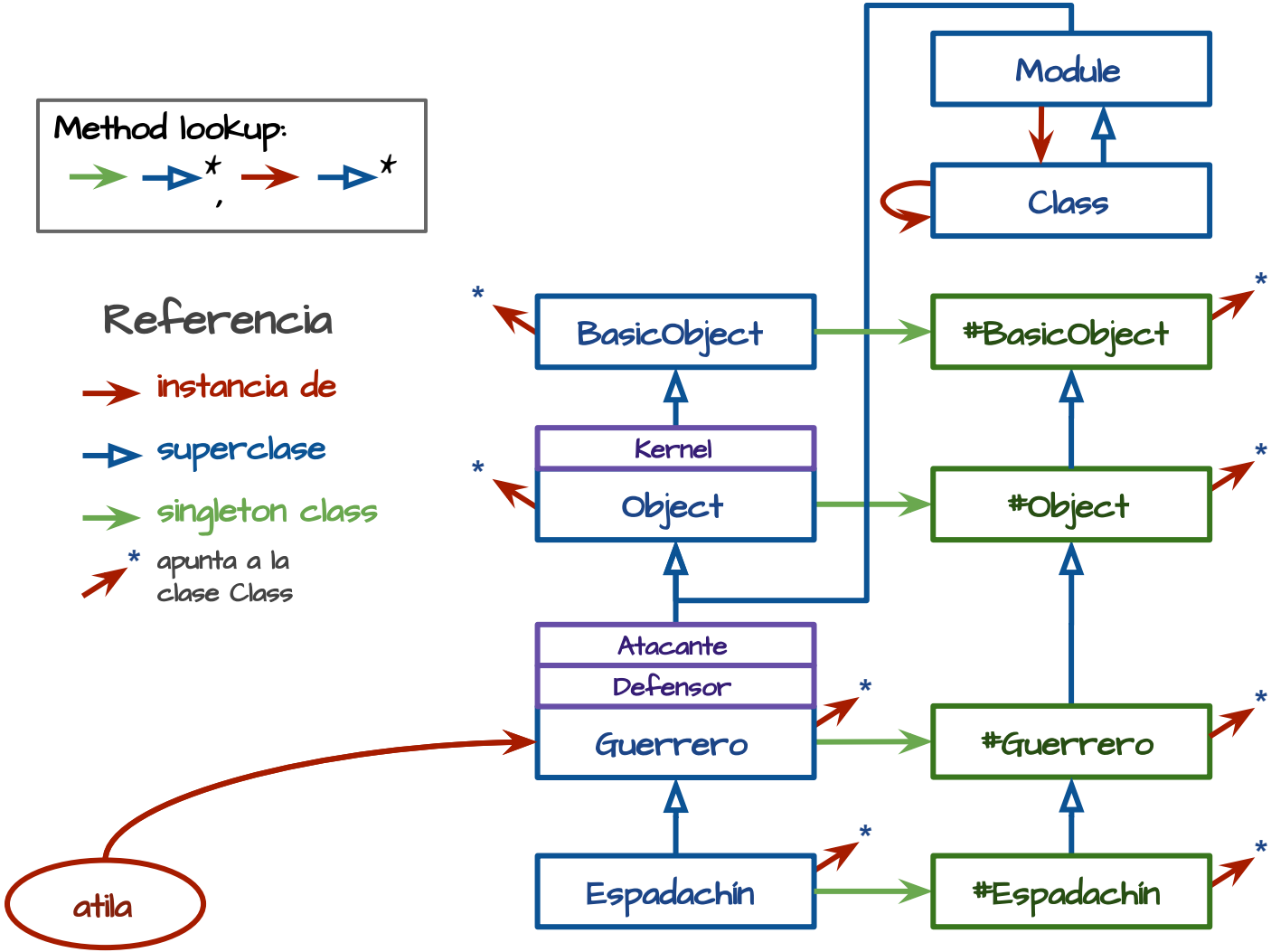

Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

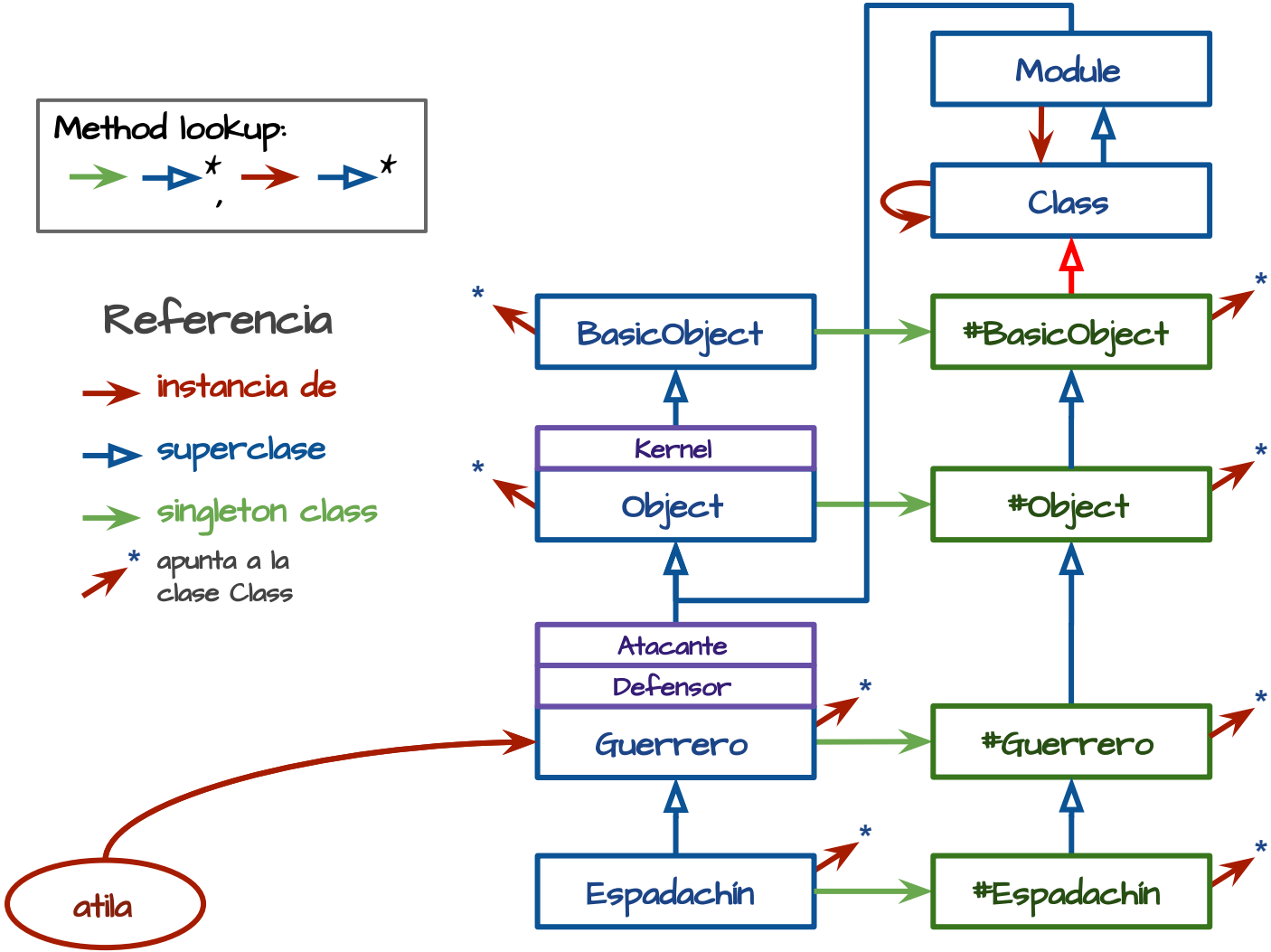

Method lookup:
*
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

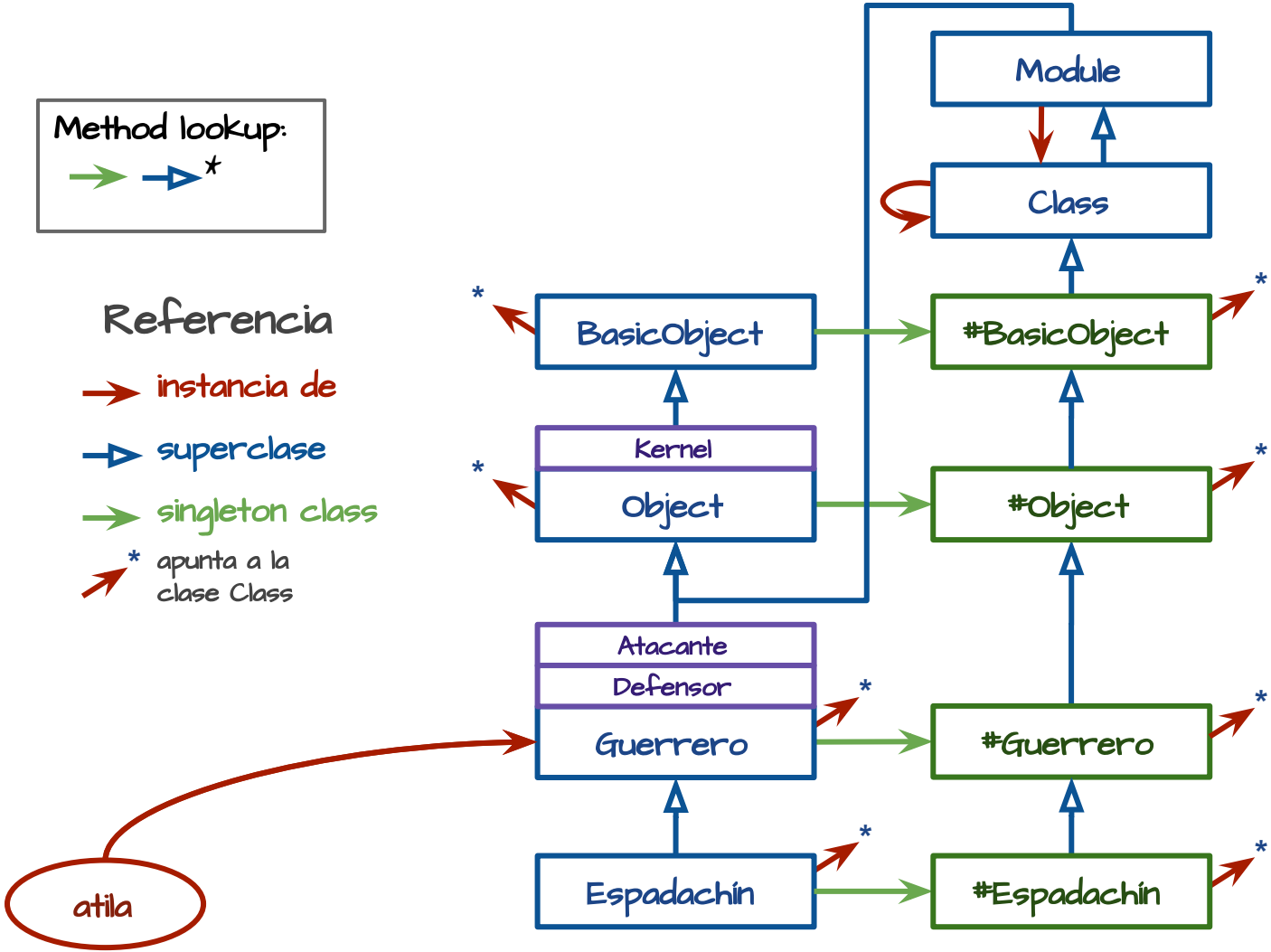

Method lookup:
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila


Method lookup:
*
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

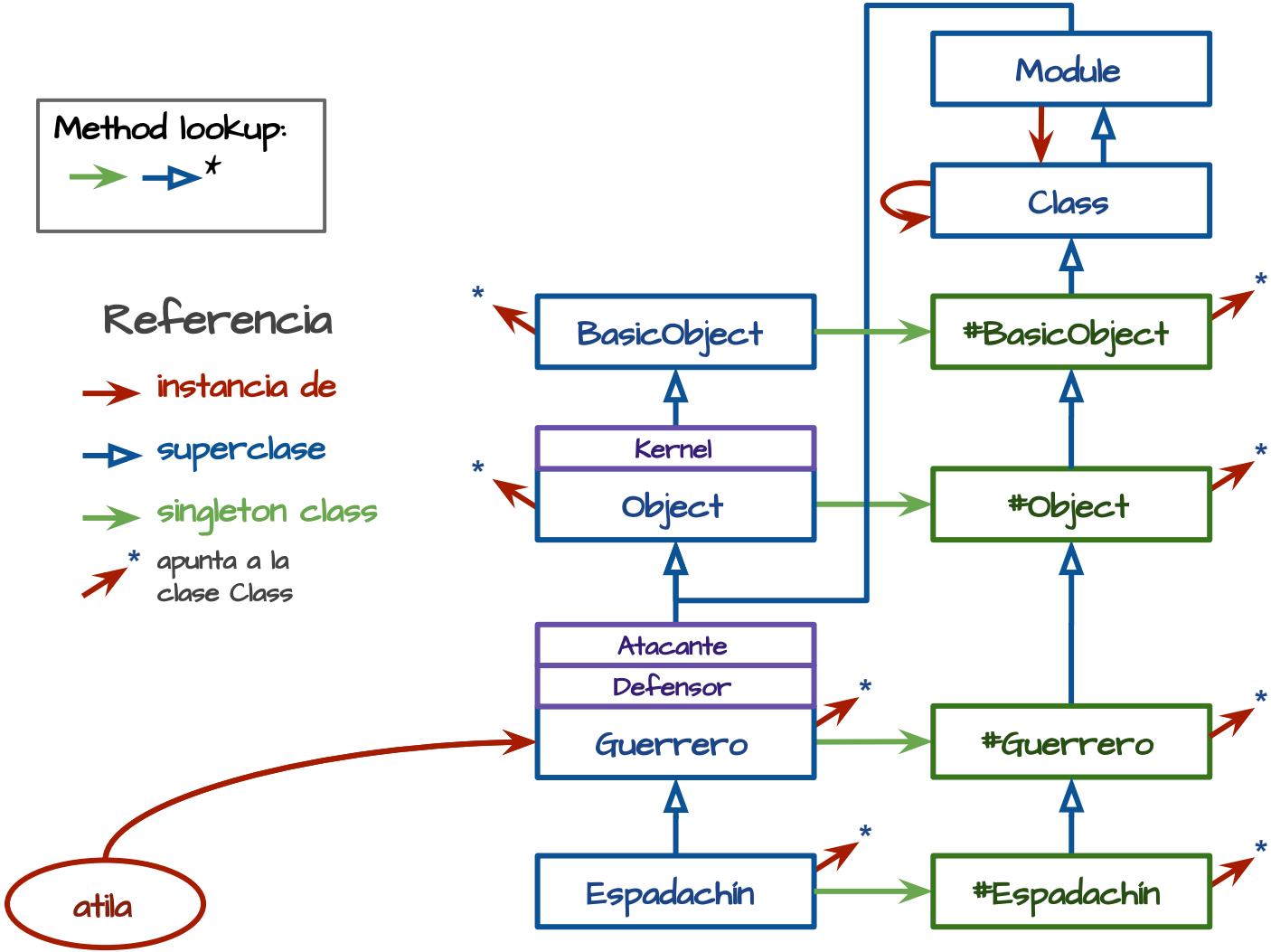
¿Podemos simplificar el *method lookup*?

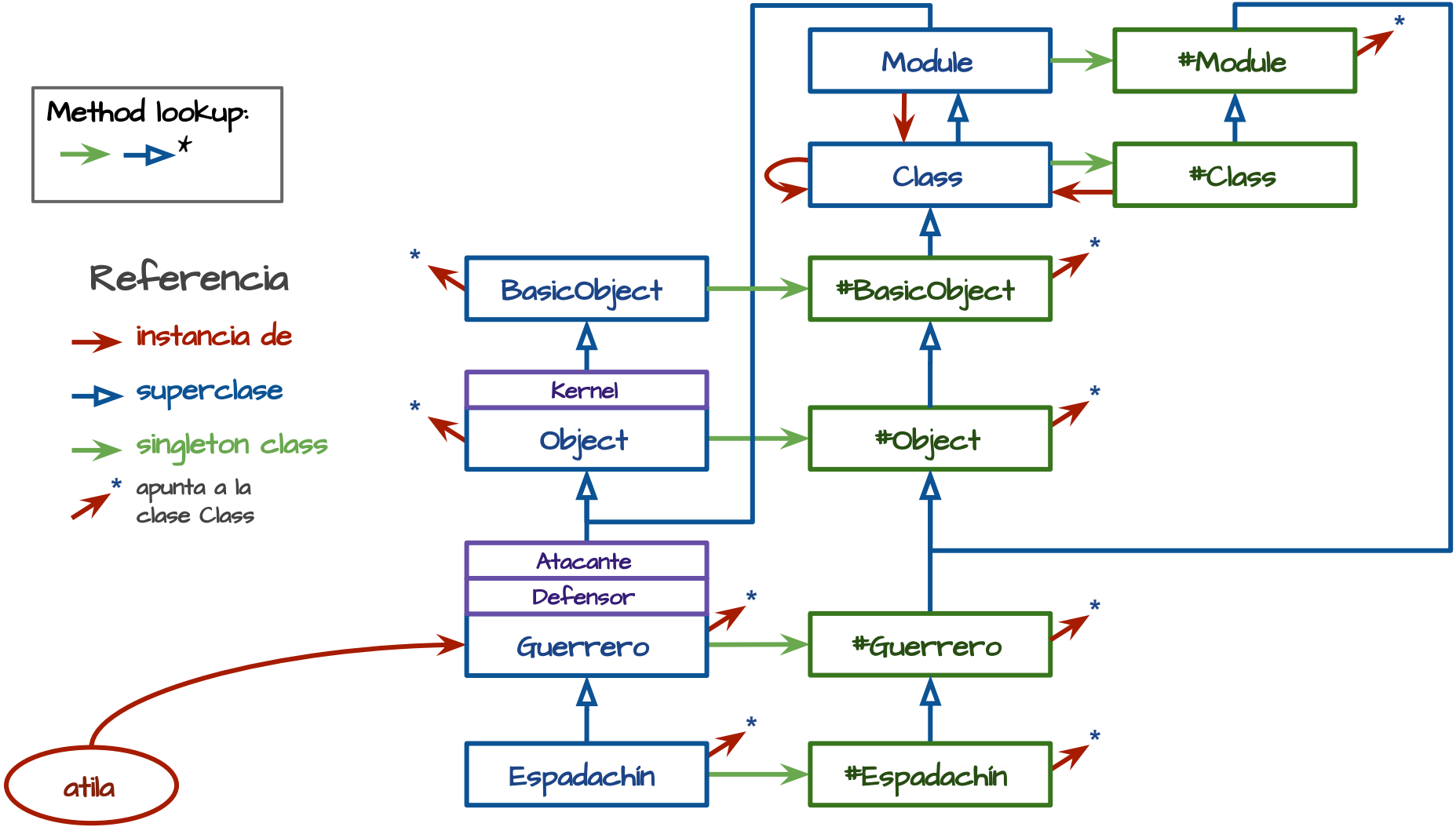

Method lookup:
*
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila


Method lookup:
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

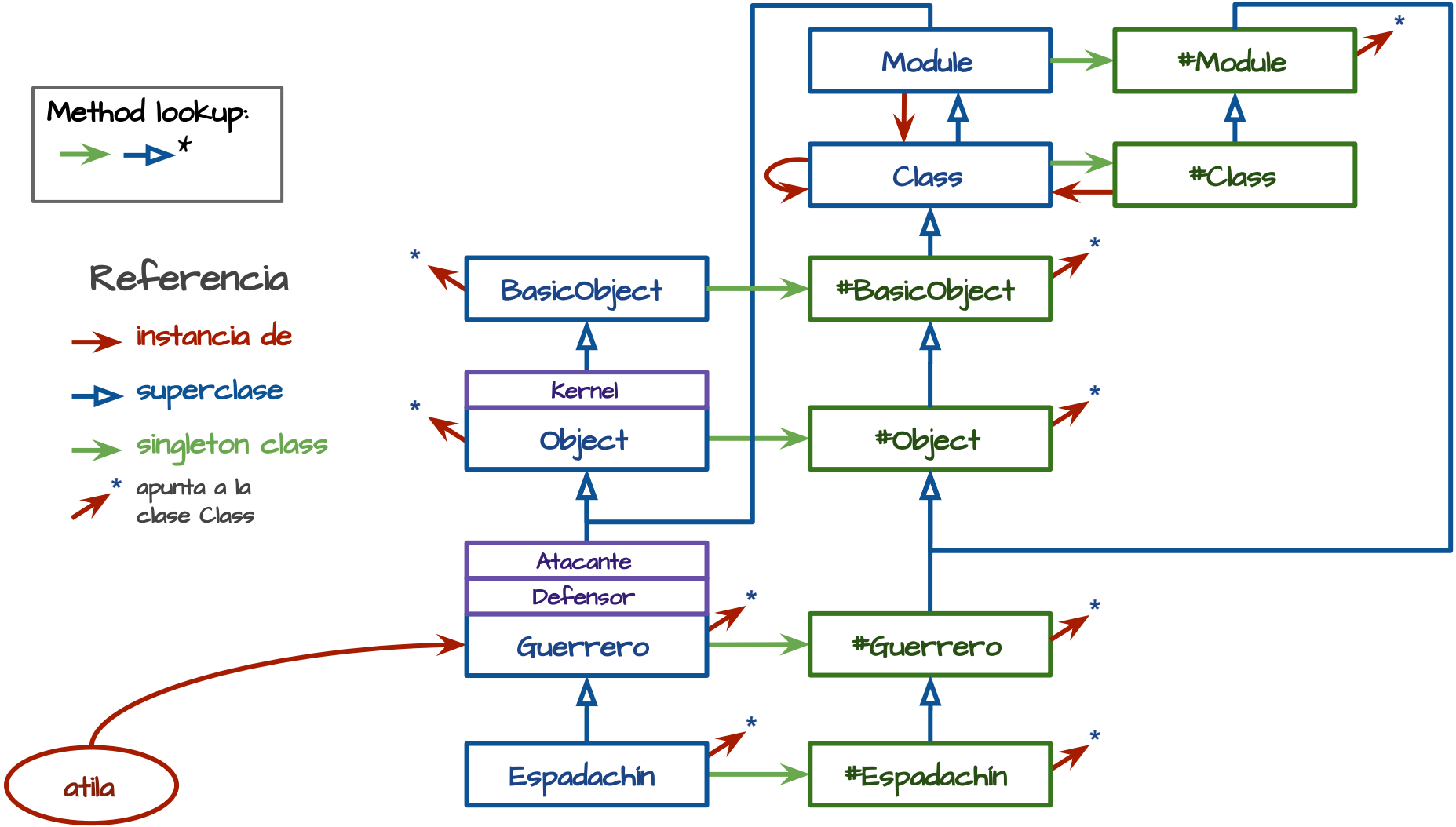

Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

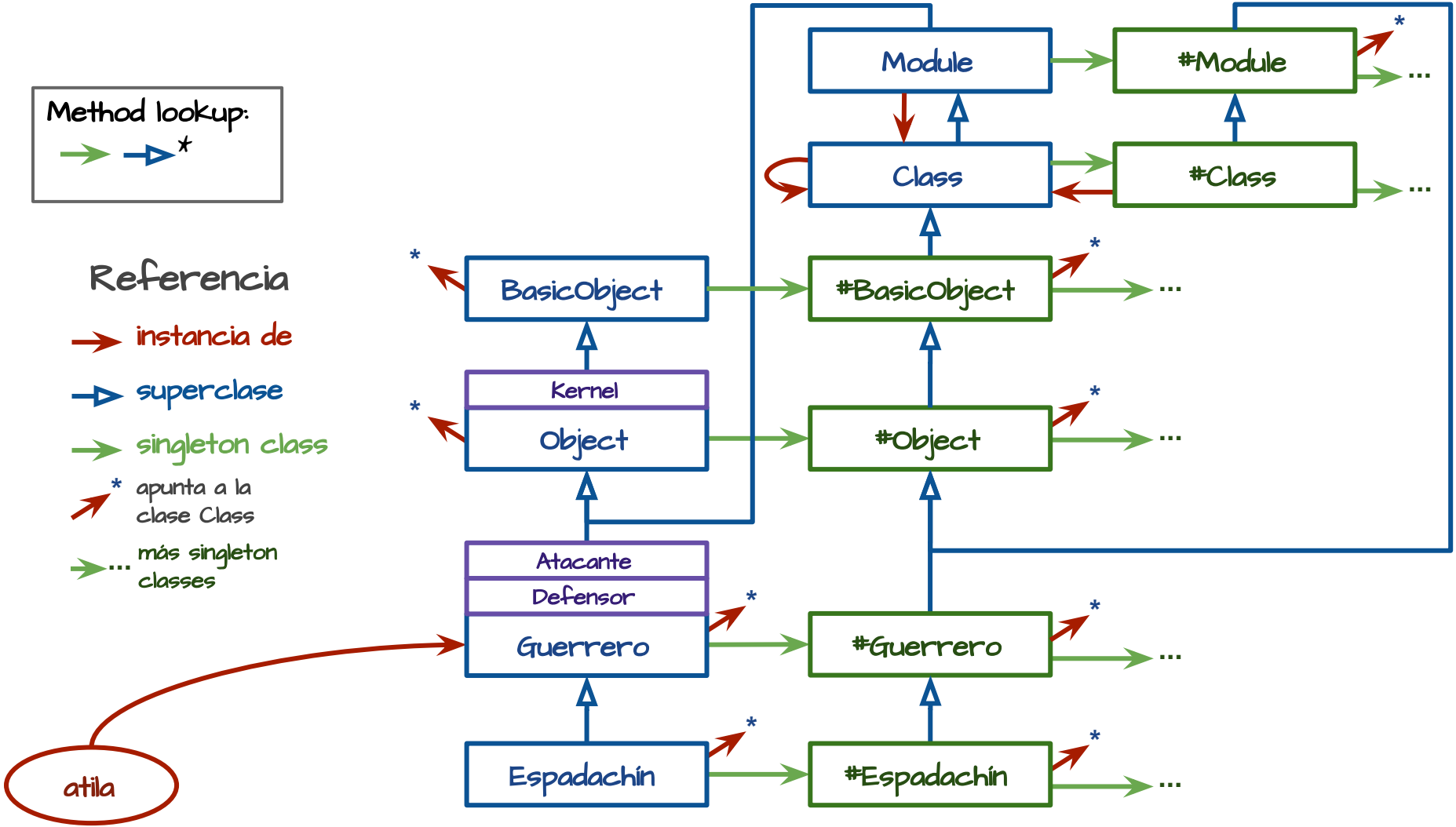
El camino a partir de la *singleton class* **agrega un prefijo** (estricto) al camino a partir de la respuesta a *class*


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

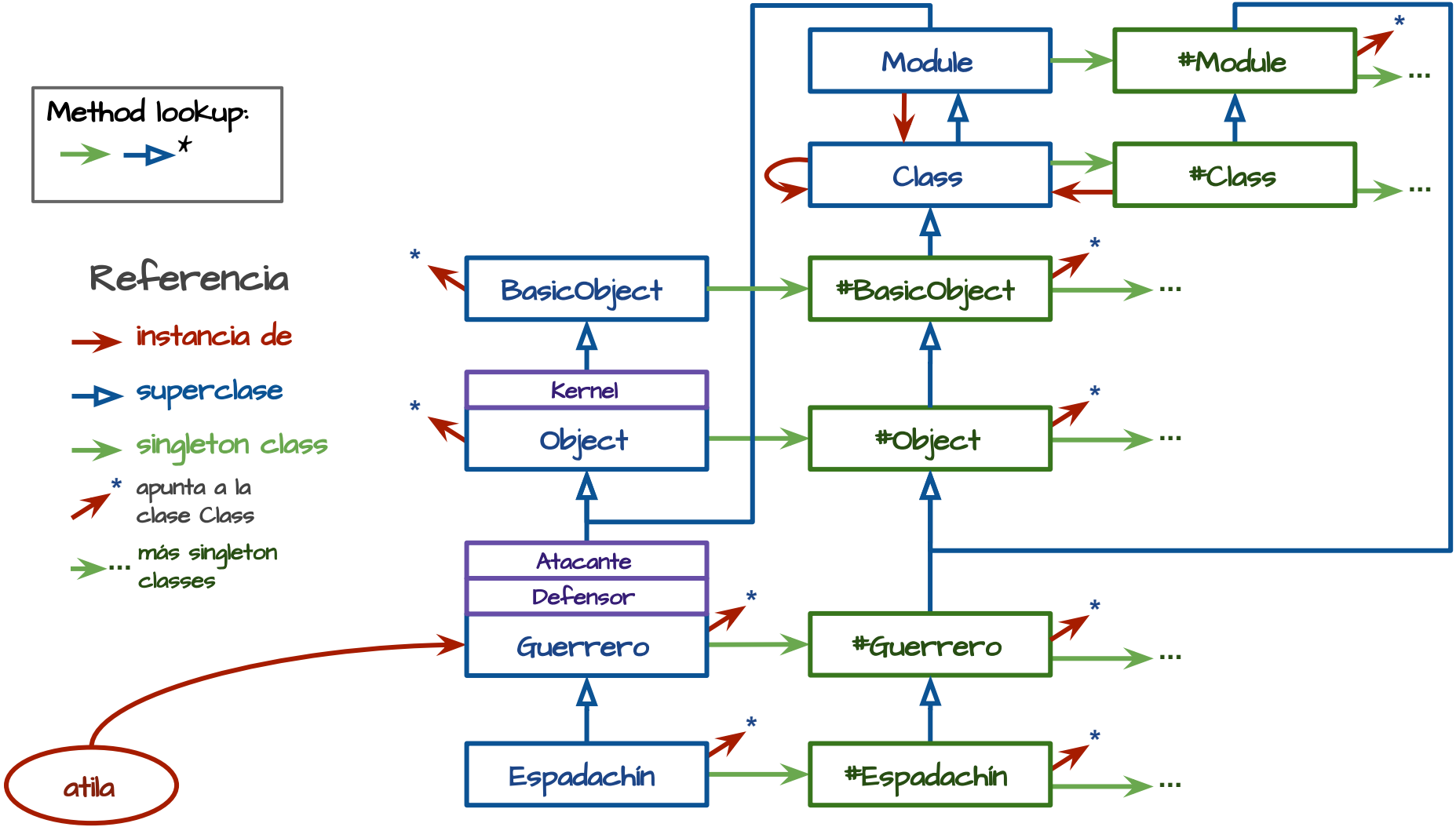

Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

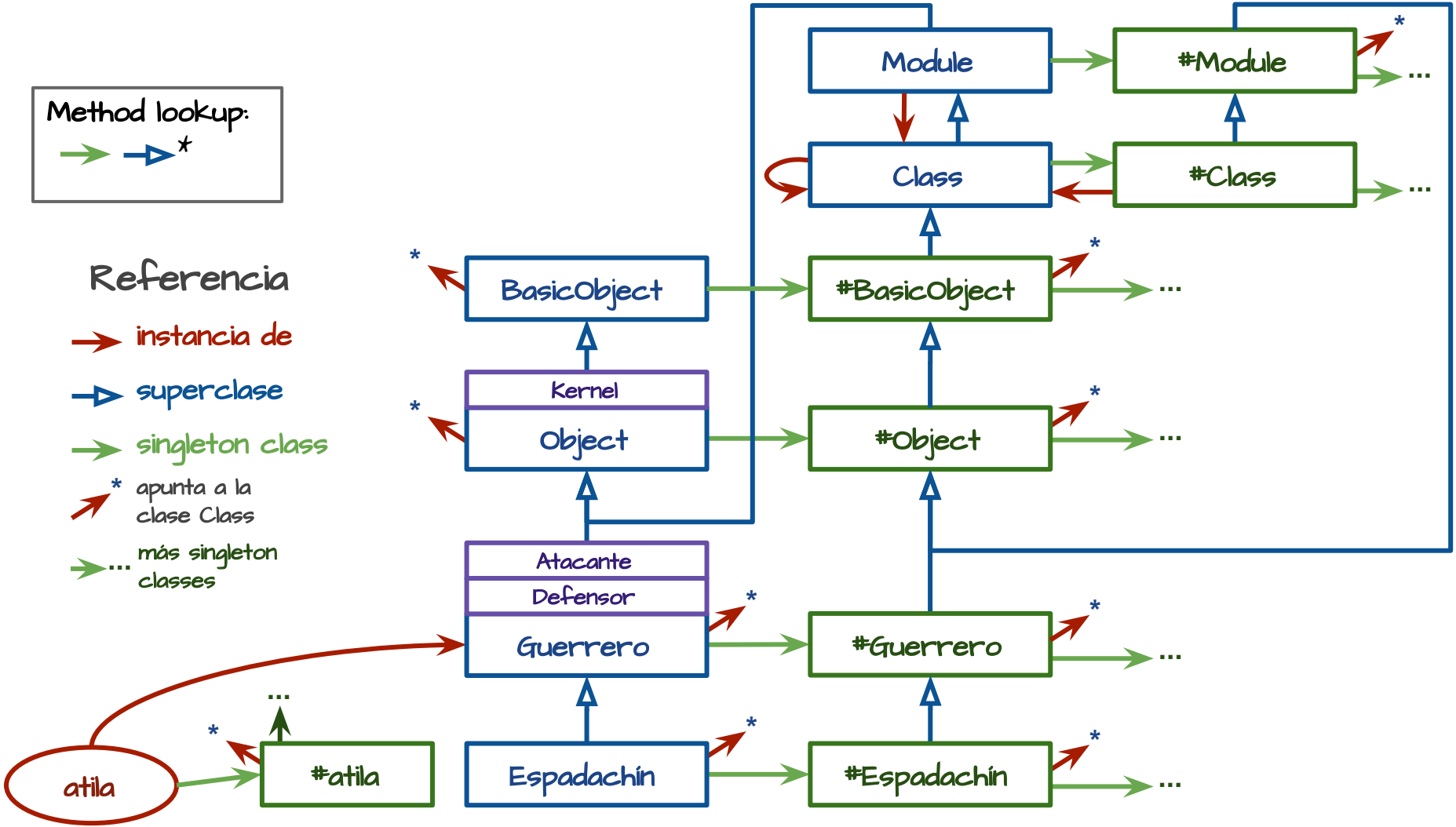
Pero... una *singleton class* también es una clase!

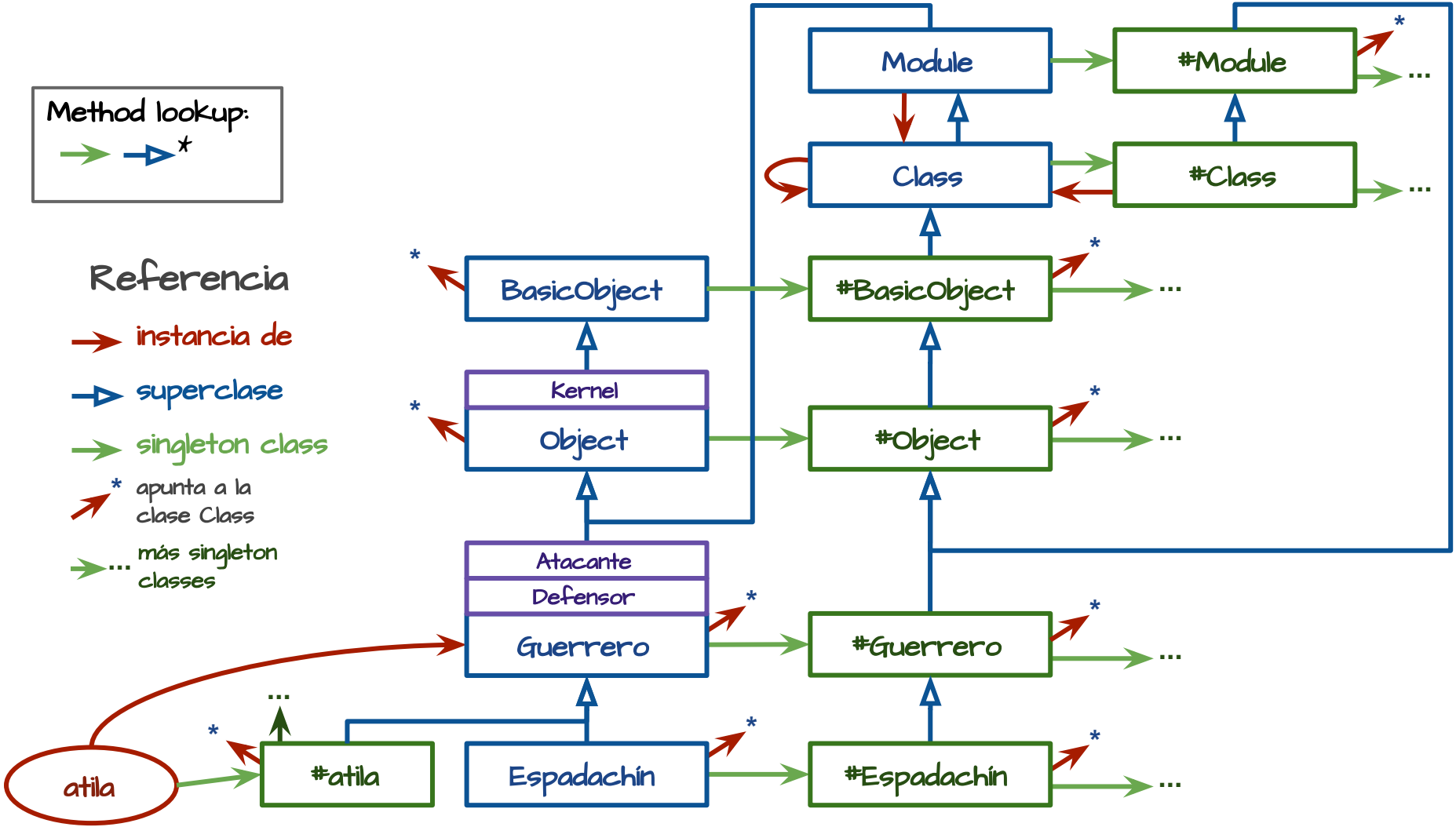

Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
... más singleton classes
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila

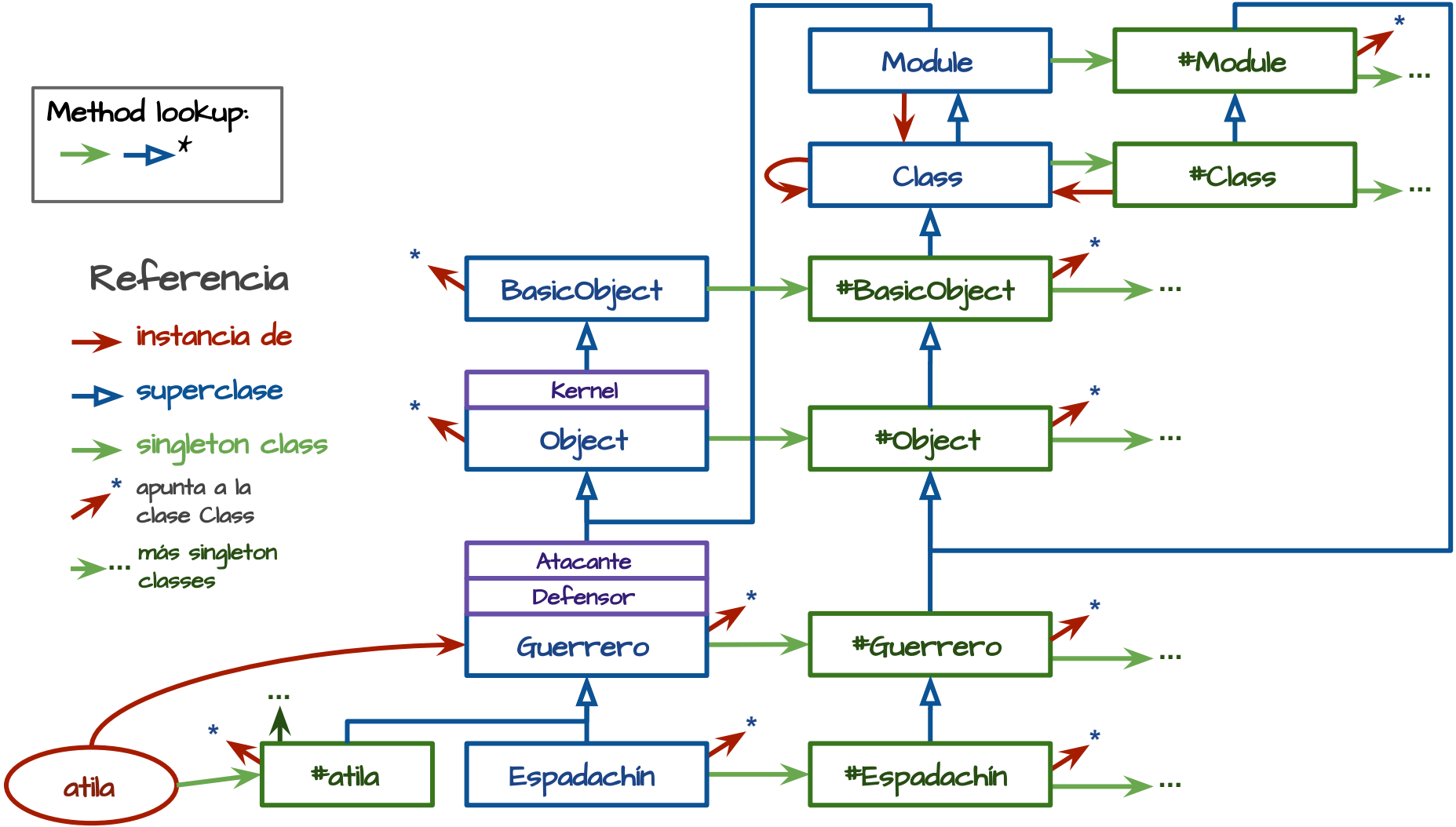
¿Qué pasa con el *method lookup* cuando un objeto (que no es una clase) recibe un mensaje?


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
... más singleton classes
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
... más singleton classes
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
Espadachín
#Espadachín
atila
#atila
*
...

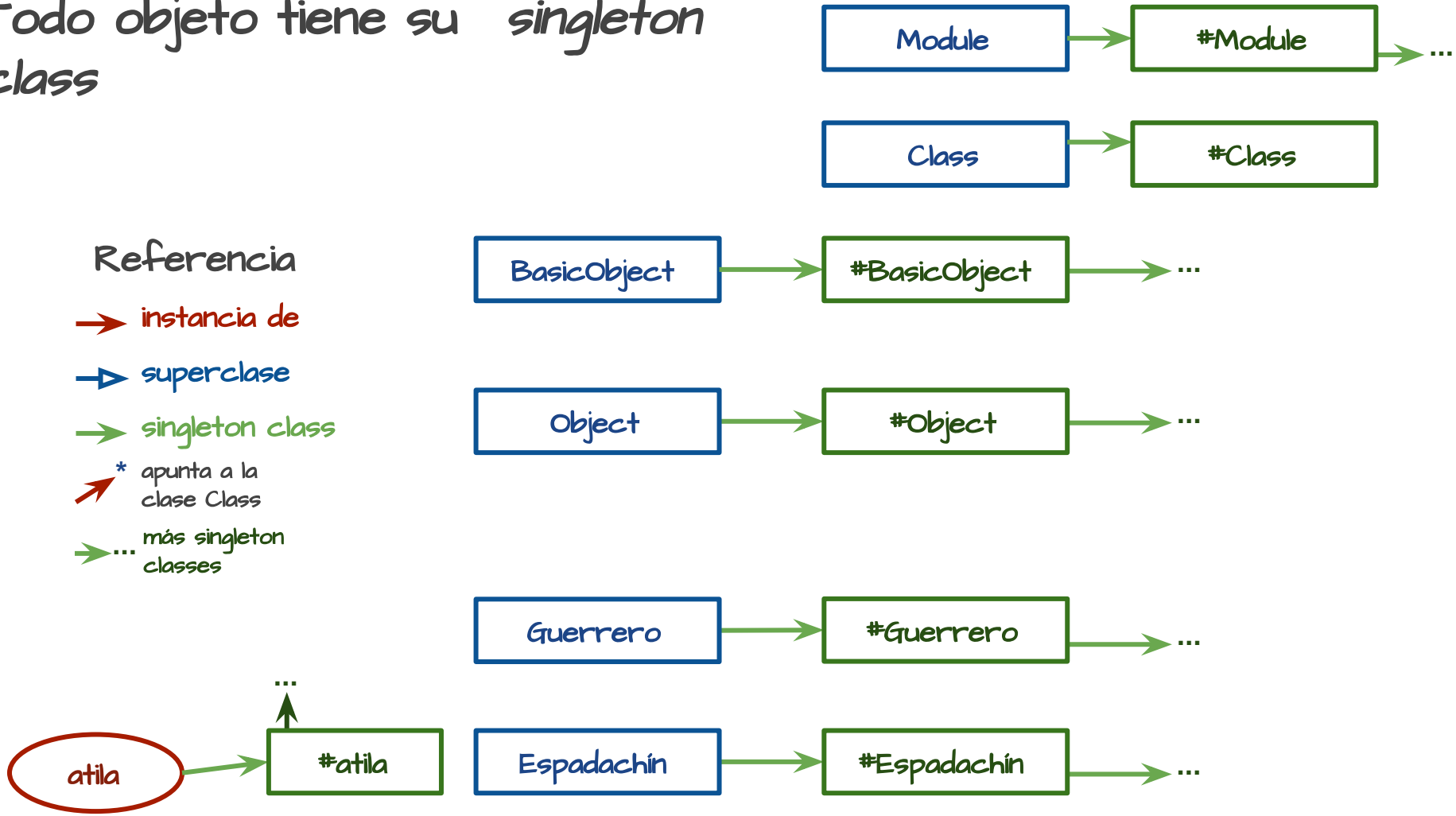

Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
... más singleton classes
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
atila
#atila
Espadachín
#Espadachín

Se sigue cumpliendo que el camino a partir de la *singleton class* **agrega un prefijo** al camino a partir de la respuesta a *class*


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
... más singleton classes
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
atila
#atila
Espadachín
#Espadachín

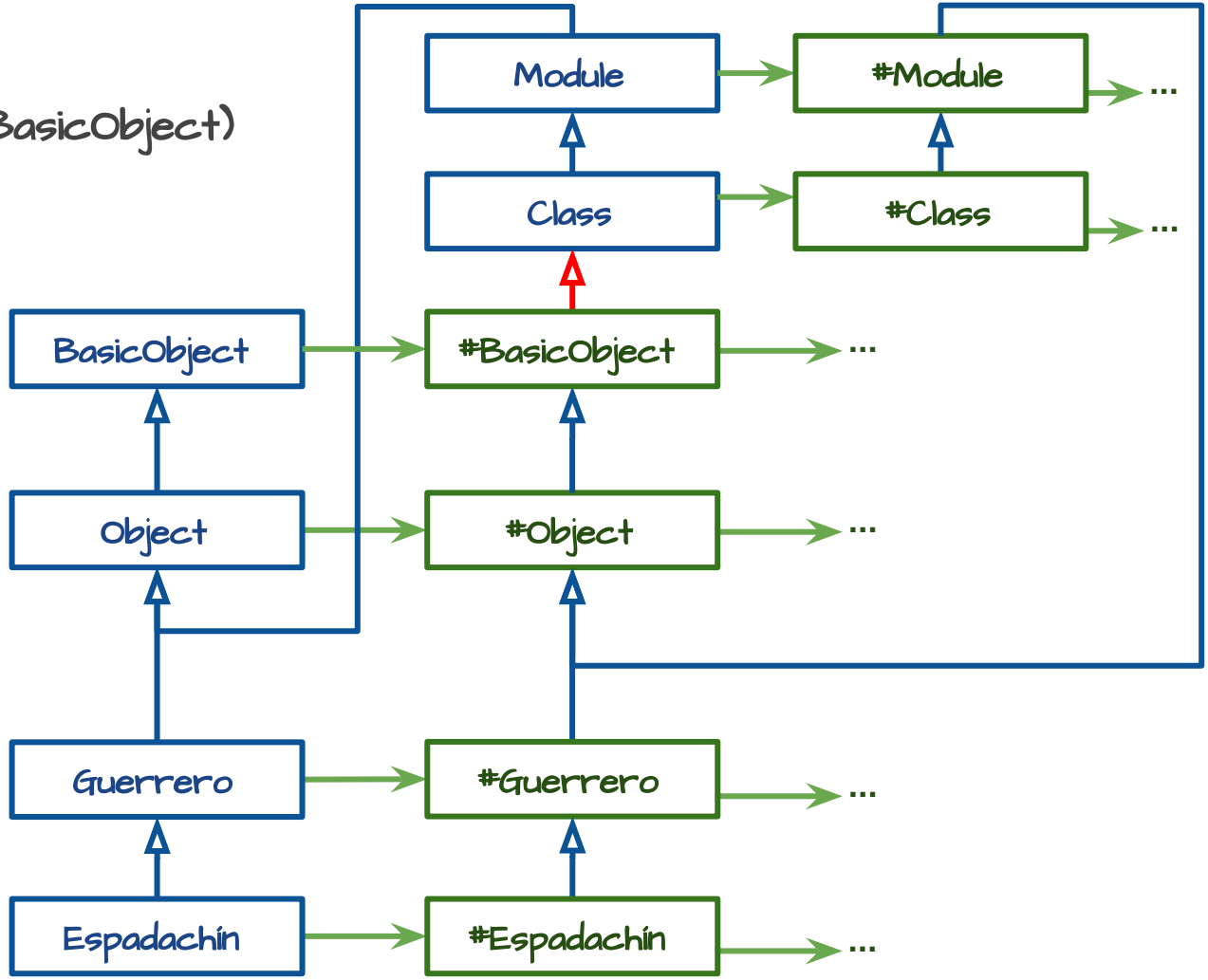
Algunas regularidades en el metamodelo

Todo objeto tiene su *singleton class*



Jerarquías paralelas

(excepto en el caso de BasicObject)



Referencia

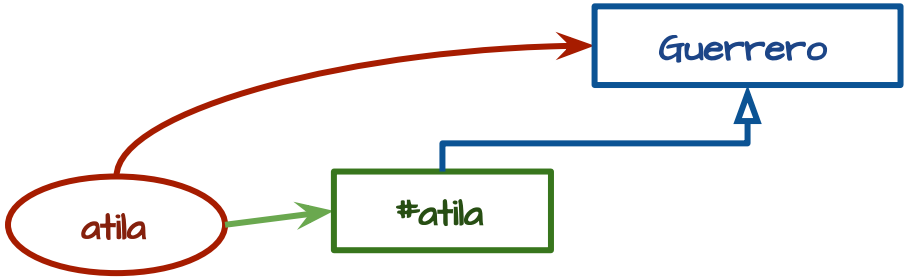
- instancia de
- superclase
- singleton class
- * apunta a la clase Class
- ... más singleton classes

Jerarquía de la singleton class de un objeto que no es una clase:

"la *superclase* de la *singleton class* es la *clase*"

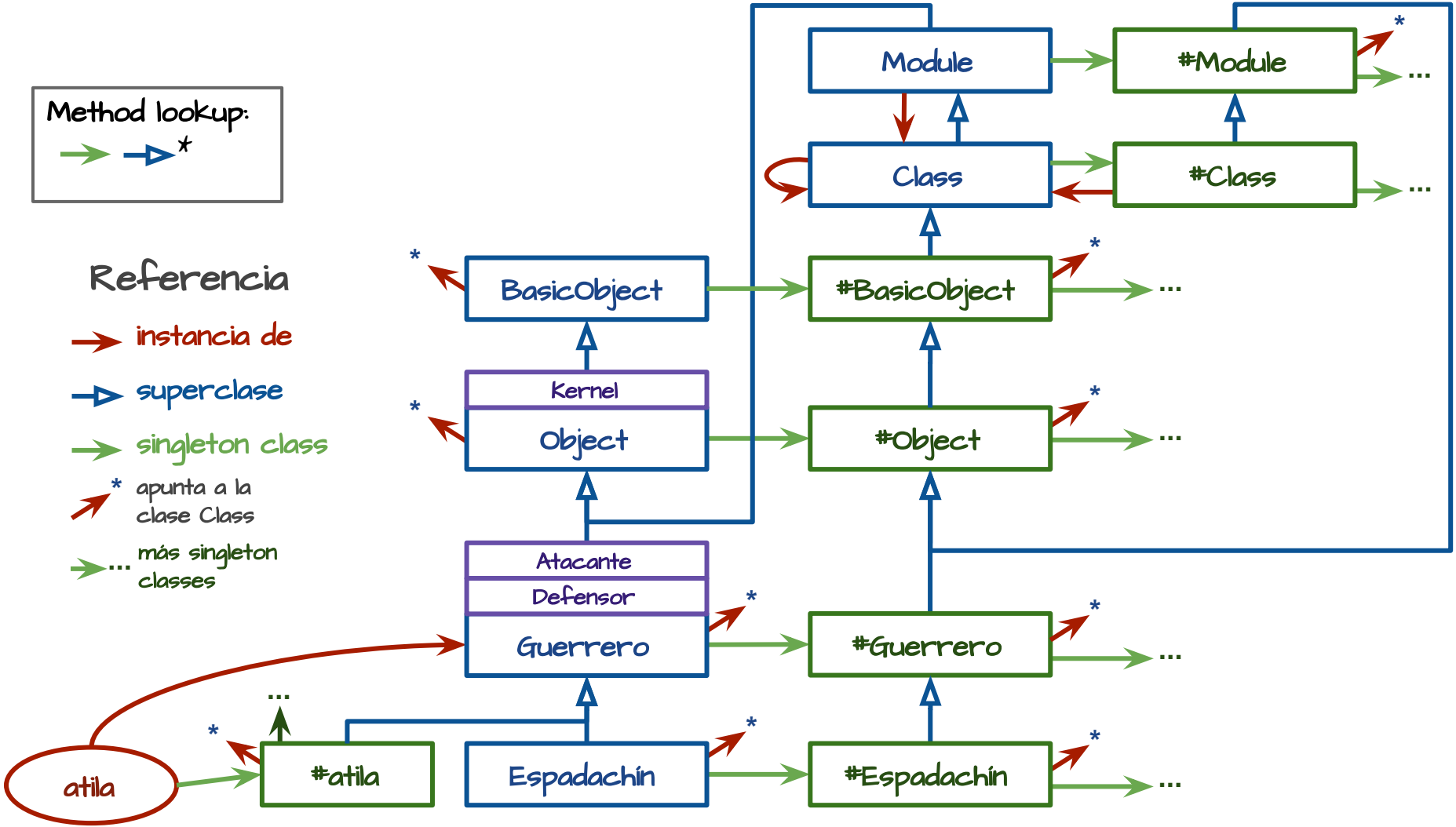
Referencia

- instancia de
- superclase
- singleton class
- * apunta a la clase Class
- ... más singleton classes



En resumen

- Todo objeto tiene una singleton class.
  - Esto incluye por supuesto a las clases, y por ende a las singleton classes.
- La superclase de la singleton class de un objeto…
  - Es la clase del objeto (si el objeto no es una clase).
  - Es la singleton class de la superclase del objeto (si el objeto es una clase).
    - Excepto para `BasicObject`.


Method lookup:
*
Referencia
instancia de
superclase
singleton class
* apunta a la clase Class
... más singleton classes
Module
#Module
Class
#Class
BasicObject
#BasicObject
Kernel
Object
#Object
Atacante
Defensor
Guerrero
#Guerrero
atila
#atila
Espadachín
#Espadachín

¿Preguntas?

Intercesión

En donde analizamos las herramientas que nos da Ruby para ejecutar y definir comportamiento dinámicamente.

¿Qué herramientas nos provee Ruby para definir comportamiento?

Clases abiertas

Mensaje *define_method*

(necesitamos pasar un *bloque*)

Comentario: ¿Clases como “moldes”?

Bloques: reificando conjuntos de colaboraciones

Ruby tiene una sintaxis un poco particular...

Instancias de *Proc* (que son objetos) vs. bloques sintácticos (que pueden enviarse como parte de un mensaje)

¿Qué objeto es *self* en el contexto de un bloque?

Familia de métodos *eval/exec*

- `instance_exec`
  Evalúa un bloque en el contexto de un objeto (es decir, `self` dentro del bloque será una referencia al objeto receptor del `instance_exec`).
  Los métodos se definen en la *singleton class* del objeto receptor del mensaje.

- `class_exec/module_exec`
  Evalúa un bloque en el contexto de una clase o un módulo (es decir, `self` dentro del bloque será una referencia al objeto receptor del `class_exec/module_exec`).
  Los métodos se definen en la clase/módulo receptor del mensaje.

¿Preguntas?